



Katedra Informatyki i Automatyki
Politechnika Rzeszowska

INŻYNIERIA OPROGRAMOWANIA

WSTĘP TEORETYCZNY DO ĆWICZEŃ LABORATORYJNYCH

dr inż. Joanna Marnik, dr inż. Grzegorz Dec

Ten utwór jest chroniony prawem autorskim. Właścicielem praw autorskich majątkowych jest Politechnika Rzeszowska. W celu uzyskania licencji na korzystanie z utworu, należy skontaktować się z Centrum Transferu Technologii PRz. Jakiegokolwiek korzystanie z utworu bez uzyskania licencji podlega odpowiedzialności karnej, zgodnie z rozdziałem 14 Ustawy o prawie autorskim i prawach pokrewnych.

Rzeszów, 25.03.2023

Spis treści

SPIS TREŚCI	2
1 DIAGRAMY PRZYPADKÓW UŻYCIA.....	4
1.1 Wstęp	4
1.2 UML.....	4
1.3 Przypadek użycia	5
1.4 Aktor	5
1.5 Interakcja (asocjacja)	6
1.6 Modelowanie przypadków użycia.....	7
1.7 Tworzenie diagramu przypadków użycia	9
1.8 Dokumentacja przypadków użycia.....	10
2 IDENTYFIKACJA PROCESÓW BIZNESOWYCH NA PODSTAWIE HISTORII UŻYTKOWNIKÓW	13
2.1 Wstęp	13
2.2 Identyfikacja ról i procesów biznesowych.....	13
2.3 Zakres usług systemu.....	16
2.4 Macierz odpowiedzialności.....	16
2.5 Makiety GUI.....	17
3 IDENTYFIKACJA STRUKTUR DANYCH NA PODSTAWIE HISTORII UŻYTKOWNIKÓW.....	18
3.1 Wstęp	18
3.2 Analiza danych z historyjek użytkowników	20
4 MODELOWANIE DANYCH ZA POMOCĄ DIAGRAMU ZWIĄZKÓW ENCJI (ERD)	30
4.1 Wstęp	30
4.2 Model związków encji (ERM) i diagram związków encji (ERD)	32
4.3 Budowa modelu danych	37

4.4	Wprowadzenie do programu Data Modeler	39
5	DIAGRAMY SEKWENCJI	42
5.1	Wstęp	42
5.2	Linia życia	42
5.3	Komunikat	43
5.4	Fragment	44
6	DIAGRAMY KLAS	46
6.1	Wstęp	46
6.2	Relacje	48
6.3	Proces tworzenia diagramu klas	49
6.4	Mapowanie obiektowo-relacyjne (ORM)	49
7	DIAGRAM HIERARCHII FUNKCJI (FHD)	53
7.1	Wstęp	53
7.2	Wprowadzenie do tworzenia diagramu FHD	54
8	DIAGRAM PRZEPŁYWU DANYCH.....	56
8.1	Wstęp	56
8.2	Rodzaje diagramów DFD	56
8.3	Elementy diagramu DFD	56
8.4	Wprowadzenie do tworzenia diagramu DFD	58
9	SZACOWANIE PRACOWNOŚCI WYKONANIA PROJEKTU	61
9.1	Wstęp	61
9.2	Metoda punktów przypadków użycia	64
	BIBLIOGRAFIA	72

1 Diagramy przypadków użycia

1.1 Wstęp

Tworzenie infrastruktury informatycznej wymaga określenia potrzeb, na jakie ma ona odpowiadać. Wobec tego pierwszym etapem działań powinna być analiza potrzeb, z której uzyskuje się wymagania budowanego systemu. W większości przypadków efekty takiej analizy mają najpierw formę zapisu słownego. Na kolejnych etapach powinny one zostać usystematyzowane i zwizualizowane. Na podstawie uzyskanych informacji następuje zdefiniowanie funkcji, które będą realizowane w ramach tworzonej infrastruktury.

Do zapisania wymagań (ang. requirements), dotyczących systemu oraz modelowania jego otoczenia, można posłużyć się diagramami przypadków użycia (ang. use case diagram). Diagram przypadków użycia pozwala przedstawić w formie graficznej związku pomiędzy aktorami i przypadkami. Aktorzy mogą być ludźmi lub innymi systemami. Na diagramie umieszcza się także wzajemne relacje między poszczególnymi przypadkami użycia.

Diagram przypadków użycia odnosi się do statycznych aspektów opisywanego z jego użyciem systemu. Wykorzystywany jest głównie do wyznaczania i modelowania zachowania systemu, w taki sposób, żeby użytkownicy mogli zrozumieć jak z niego korzystać, a programiści mogli go zaimplementować. Dzięki diagramom przypadków użycia systemy, podsystemy i klasy stają się bardziej przystępne i zrozumiałe.

Model w postaci diagramu przypadków użycia jest przejrzystą formą reprezentacji wymagań wstępnych dla konstruowanego systemu. Obrazuje funkcje widoczne z zewnątrz systemu. Można go doprecyzowywać i uszczegóławiać poprzez dodawanie nowych aktorów, nowych przypadków użycia i powiązań pomiędzy nimi. Diagram przypadków użycia dostarcza bardzo abstrakcyjnego poglądu na system, z pozycji aktorów, którzy go używają. Nie włącza szczegółów, co pozwala wnioskować o systemie na odpowiednio ogólnym, abstrakcyjnym poziomie.

Każdą interakcję z systemem, przedstawioną w postaci przypadku użycia, można dodatkowo opisać, albo w postaci opisu tekstowego, albo jednym lub więcej modelem graficznym, takim jak sekwencja UML lub wykresy stanowe.

1.2 UML

UML (zunifikowany język modelowania, od ang. Unified Modeling Language) to język modelowania, wykorzystywany do tworzenia modeli systemów oprogramowania. Jest zbiorem 13 różnych typów diagramów. Wyłonił się z prowadzonych w latach 90. XX w. prac nad modelowaniem obiektywnym, gdy zintegrowano podobne systemy notacji obiektowej w celu utworzenia UML. Znacząca zmiana (UML 2) została ukończona w 2004 r. UML jest powszechnie akceptowany jako standardowe podejście do projektowania modeli systemów oprogramowania. Zostały również zaproponowane odmiany do bardziej ogólnego modelowania systemów, takie jak SysML [1].

Do szybkiego tworzenia diagramów w języku UML można wykorzystać gotowe narzędzia, na przykład UMLet, lub jego wersję obsługiwaną z pozycji przeglądarki internetowej – UMLetino [2], rozpowszechniane na licencji GNU.

UMLet posiada prosty interfejs użytkownika. Może eksportować diagramy do zdjęć, formatów wektorowych (SVG) oraz formatów dokumentów (PDF). Pozwala na używanie schowka do

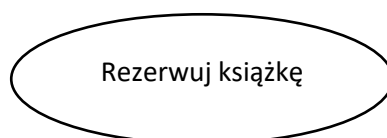
kopiowania i wklejania diagramów do innych aplikacji jako zdjęć. Wspierane jest także tworzenie własnych elementów UML.

1.3 Przypadek użycia

Zgodnie z definicją, przypadek użycia w UML jest zdefiniowany jako zbiór akcji wykonywanych przez system, które powodują efekt zauważalny dla aktora (zob. p. 1.4). Przypadki użycia zawsze muszą być inicjowane (bezpośrednio lub przez zależności) przez aktora i wykonywane w jego imieniu. Ponadto, przypadek użycia musi dostarczać pewną wartość użytkownikowi oraz musi być kompletny, to znaczy w pełni realizować podaną funkcjonalność oraz dostarczać wyniki aktorowi. Zazwyczaj przypadki użycia są krótkimi frazami bądź zdaniami w trybie rozkazującym, opisującymi część funkcji oferowanych użytkownikowi przez system.

Przypadek użycia charakteryzuje się unikalną nazwą, np. „Analiza danych”, „Dokonanie zakupu”, odnoszącą się jednoznacznie do funkcji, którą prezentuje.

Podobnie jak inne elementy UML, przypadki użycia są często grupowane w pakietach i można się do nich odwoływać za pomocą nazwy kwalifikowanej. Z nazwą przypadku użycia skojarzona jest

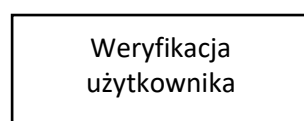


Rys. 1. Przypadek użycia

sekwencja zdarzeń opisujących działanie systemu po wywołaniu przypadku użycia. Symbolem graficznym przypadku użycia jest elipsa narysowana ciągłą linią oraz nazwa. Przykład przypadku użycia jest pokazany na Rys. 1.

Zadaniem przypadków użycia jest modelowanie oczekiwanego zachowania systemu. Modelując zachowanie za pomocą przypadków użycia nie ma konieczności zagłębiania się w zagadnienia związane z implementacją tego zachowania. Przypadki użycia są przewidziane do przekazywania informacji o wymaganych funkcjach, od końcowych użytkowników do kierowników projektów i programistów.

Diagram przypadków użycia może zawierać także bloki ponownego użycia, reprezentujące fragment



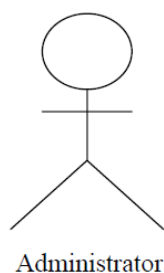
Rys. 2. Blok ponownego użycia

systemu, który jest używany przez kilka przypadków. Blok taki określony jest nazwą umieszczoną w prostokącie. Przykładowy blok ponownego użycia pokazany jest na Rys. 2.

1.4 Aktor

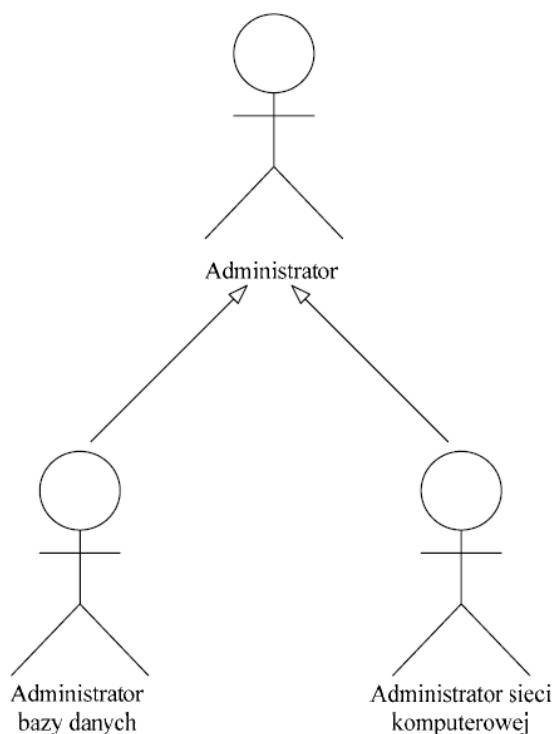
Elementami nie będącymi częścią systemu, ale ściśle z nim związanymi, są aktorzy. Aktor jest rolą, jaką użytkownik przyjmuje podczas wykonywania przypadku użycia. Aktorom nadaje się unikalne nazwy. Ponieważ użytkownik może pełnić wiele ról, pojedynczy użytkownik może być reprezentowany przez wielu aktorów. Podobnie, pojedynczy aktor może reprezentować wielu użytkowników. Dobrze opracowanie modelu aktorów pozwala na zrozumienie kto lub co będzie

wchodziło w interakcję z systemem. Aktor jest reprezentowany jako schematyczna postać, z nazwą umieszczoną pod spodem, jak pokazano na Rys. 3.



Rys. 3. Aktor

Aktor może być zwykłym klasycznym użytkownikiem (człowiekiem), może to być też program, urządzenie sprzętowe czy też system zewnętrzny. Niektórzy aktorzy mogą stanowić formy specjalizacji innych aktorów – pewna funkcja jednego aktora może wchodzić w zakres obowiązków innego aktora. Stosując uogólnienia można definiować ogólne rodzaje aktorów (np. Administrator) i uszczegółowienia (np. Administrator bazy danych, Administrator sieci komputerowej). Przykład uszczegółowienia aktora pokazany jest na Rys. 4.

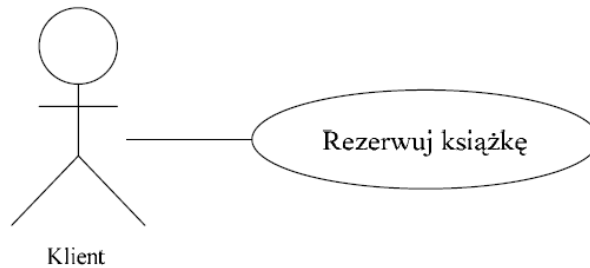


Rys. 4. Uszczegółowienie aktorów

Aktorzy pomagają określić granice pomiędzy elementami, które należy zaimplementować w modelowanym systemie, a elementami istniejącymi poza systemem.

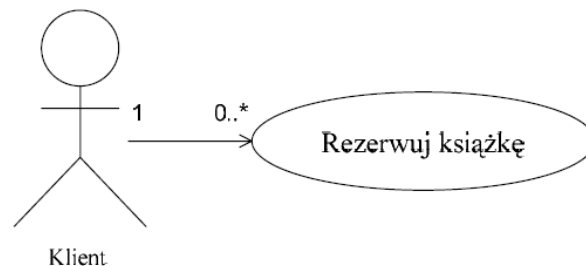
1.5 Interakcja (asocjacja)

Przypadki użycia komunikują się z aktorami poprzez powiązania, pokazujące, który aktor ma dostęp do podanego przypadku użycia. Powiązanie nazywane jest asocjacją (interakcją), przedstawianą na diagramie linią ciągłą, łączącą aktora z przypadkiem użycia. Interakcja skierowana opisuje sytuację, w której obiekt inicjuje przypadek. Przykładowy związek asocjacji pokazany jest na Rys. 5.



Rys. 5. Związek asocjacji w diagramach użycia

Diagramy przypadków użycia pozwalają na określenie liczebności (ang. multiplicity) końcówek związku asocjacji oraz kierunku asocjacji – nawigacji. Kierunek asocjacji, pokazujący kto jest stroną inicjującą interakcję, wskazuje strzałka. Przykładowy związek asocjacji, z określoną liczebnością i nawigacją przedstawiony jest na Rys. 6.



Rys. 6. Związek asocjacji z określoną liczebnością i nawigacją

1.6 Modelowanie przypadków użycia

Przypadki użycia często są związane z innymi przypadkami użycia, należącymi do tego samego systemu. Związki te mogą być modelowane jako:

- zawieranie,
- rozszerzenie,
- uogólnienia (generalizacja) lub
- realizacja.

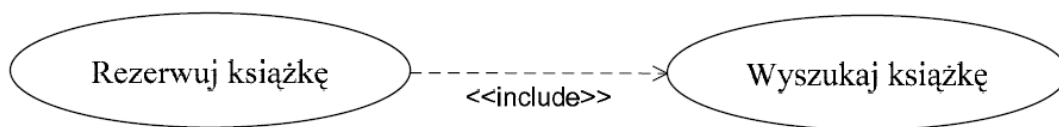
1.6.1 Zawieranie przypadków użycia

Związek zawierania między przypadkami użycia polega na tym, że bazowy przypadek użycia jawnie włącza zachowanie innego przypadku użycia, w miejscu przez siebie określonym. Włączany przypadek nigdy nie występuje samodzielnie – jego egzemplarze mogą być tylko częścią większego, zawierającego go przypadku użycia. Związku zawierania używa się w celu uniknięcia wielokrotnego opisywania tego samego ciągu zdarzeń.

Zależność zawierania na diagramie przypadków użycia występuje w dwóch sytuacjach:

- istnieje możliwość wydzielenia w formie zawieranego przypadku użycia pewnej spójnej, wspólnej dla kilku innych przypadków użycia funkcjonalności;
- interakcje aktor–system, wyrażone w dokumentacji scenariusza tego przypadku użycia, są bardzo liczne.

Na diagramie zależność zawierania oznacza się w sposób pokazany na Rys. 7.



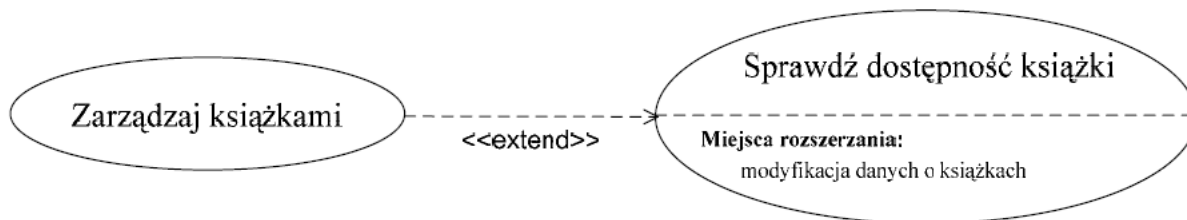
Rys. 7. Zależność zawierania przypadków użycia

Zależność zawierania jest skierowana od przypadku zawierającego do zawieranego, i oznaczana przez `<<include>>`.

1.6.2 Rozszerzanie przypadków użycia

Związek rozszerzania między przypadkami użycia polega na tym, że bazowy przypadek użycia, w sposób domniemany, włącza zachowanie innego przypadku, w miejscu określonym pośrednio, przez rozszerzający przypadek użycia. Bazowy przypadek użycia może wystąpić samodzielnie, ale pod pewnymi warunkami jego zachowanie może być rozszerzone przez zachowanie innego przypadku użycia. Związek rozszerzania służy do modelowania fragmentów przypadku użycia, postrzeganych przez użytkownika jako opcjonalne zachowanie systemu. Tworzenie zależności rozszerzania znajduje uzasadnienie, o ile funkcjonalność reprezentowana przez rozszerzany przypadek użycia ma zostać uzupełniona o kilka dodatkowych kroków. Nie mają one być jednak automatycznie wykonywane przy każdym odwołaniu do tego przypadku użycia, a jedynie w określonych sytuacjach.

Zależność rozszerzania jest skierowana od przypadku rozszerzającego do rozszerzanego i oznaczane przez `<<extend>>`. Warunki jakie muszą być spełnione, aby przypadek bazowy mógł być rozszerzony przez przypadek rozszerzający określa się w przypadku bazowym jako miejsce rozszerzania. Na Rys. 8 pokazana jest zależność rozszerzania przypadku użycia *Sprawdź dostępność książki* o przypadek rozszerzający *Zarządzaj książkami*.



Rys. 8. Zależność rozszerzania przypadków użycia

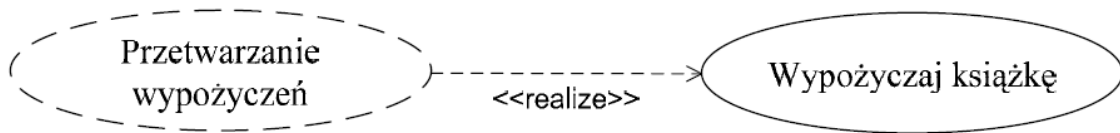
1.6.3 Uogólnienie (dziedziczenie) przypadków użycia

Uogólnienie (dziedziczenie, *ang. inheritance*) przypadku użycia zachowuje się identycznie jak uogólnienie aktorów. Specjalizowany przypadek użycia dziedziczy działanie oryginalnego przypadku użycia. Może on zamienić lub rozszerzyć to działanie, ale tylko w ramach założeń oryginalnego przypadku użycia. Za pomocą związku uogólniania (dziedziczenia) można wyrazić zależności o znacznie wyższym stopniu złożoności, poprzez wskazanie dalszych elementów dziedziczących. Dziedziczenie oznaczane jest w postaci podwójnej linii przerywanej, z oznaczeniem `<<inheritance>>`.

1.6.4 Realizacja przypadków użycia

Ponieważ przypadki użycia opisują wymagania na poziomie funkcjonalnym, język UML zapewnia mechanizmy śledzenia wymagań funkcjonalnych aż do ich implementacji. Odzworowanie to jest

nazywane *realizacją przypadku użycia*. Związki realizacji w diagramach przypadków użycia pozwalają modelować relacje pomiędzy ogólnym, funkcjonalnym opisem systemu, w postaci diagramów przypadków użycia, a jego wdrożeniem. W ten sposób diagramy przypadków użycia zostają jawnie powiązane z innymi diagramami, które zawierają informacje niezbędne do jego rozwinięcia i implementacji. Modele realizacji przypadków użycia nazywane są **współdziałaniami**. Element współdziałania na diagramie przedstawia się w formie elipsy, rysowanej linią przerywaną (Rys. 9). Jest on zazwyczaj połączony z jednym lub więcej diagramów UML.



Rys. 9. Realizacja przypadku użycia

1.7 Tworzenie diagramu przypadków użycia

Proces tworzenia diagramu przypadków użycia poprzedzony jest określeniem dwóch elementów

- Wymagań systemu i
- Funkcji systemu.

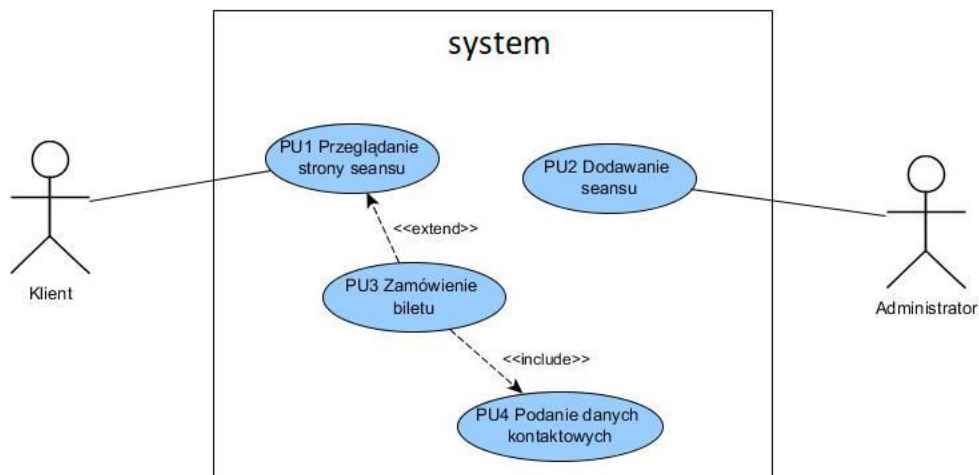
Główne etapy tworzenia diagramu przypadków użycia to:

- Określenie systemu oraz otoczenia,
- Rozpoznanie aktorów,
- Określenie interakcji pomiędzy aktorami a systemem,
- Zdefiniowanie przypadków oraz relacji pomiędzy nimi.

Dużą wagę odgrywa określenie szczegółowości diagramu, ponieważ model zbyt szczegółowy utrudnia analizę, natomiast zbyt ogólny nie pozwala na wykrycie bloków ponownego użycia.

Określenie przypadków użycia i relacji pomiędzy nimi powinna poprzedzać analiza funkcji wspólnych systemu. Funkcje takie warto jest wyodrębnić jako osobny przypadek użycia, powiązany z przypadkami użycia korzystającymi z jego funkcjonalności za pomocą relacji zawierania (*include*).

Definiując przypadek użycia jako rozszerzenia innego przypadku użycia należy pamiętać, że podstawowy (bazowy) przypadek użycia powinien być w stanie wystąpić samodzielnie, ale pod pewnymi warunkami jego zachowanie może zostać rozszerzone o inny przypadek użycia.



Rys. 10. Przykładowy diagram przypadków użycia

Przykładowy diagramu przypadków użycia pokazany jest na Rys. 10.

Budując diagram przypadków użycia warto stosować się do następujących zaleceń:

- Dla każdego aktora, znajdź funkcje (zadania), które powinien wykonywać w związku z jego działalnością w zakresie zarówno dziedziny przedmiotowej, jak i wspomagania działalności systemu informacyjnego.
- Staraj się powiązać w jeden przypadek użycia zespół funkcji realizujących podobne cele. Unikaj rozbicia jednego przypadku użycia na zbyt wiele podprzypadków.
- Nazwy dla przypadków użycia powinny być krótkie, ale jednoznacznie określające charakter zadania lub funkcji. Nazwy powinny odzwierciedlać czynności z punktu widzenia aktorów, a nie systemu, np. "wpłacanie pieniędzy", a nie "przyjęcie pieniędzy od klienta".
- Opisz przypadki użycia przy pomocy zdań w języku naturalnym, używając terminów ze słownika.
- Uporządkuj aktorów i przypadki użycia w postaci diagramu.
- Niektóre z powstałych w ten sposób przypadków użycia mogą być mutacjami lub szczególnymi przypadkami innych przypadków użycia. Przeanalizuj powiązania aktorów z przypadkami użycia i ustal, które z nich są zbędne lub mogą być uogólnione.
- Wyodrębnij "przypadki bazowe", czyli te, które stanowią istotę zadań, są normalnym, standardowym użyciem. Pomiń czynności skrajne, wyjątkowe, uzupełniające lub opcjonalne. Nazwij te przypadki bazowe. Ustal powiązania przypadków bazowych z innymi przypadkami, poprzez ustalenie ich wzajemnej zależności: sekwencji (zawierania) czy alternatywy (rozszerzenia).
- Dodaj zachowania skrajne, wyjątkowe, uzupełniające lub opcjonalne. Ustal powiązanie przypadków bazowych z tego rodzaju zachowaniem.
- Staraj się, aby bloki specyfikowane wewnątrz każdego przypadku użycia nie były zbyt ogólne lub zbyt szczegółowe. Zbyt szczegółowe bloki utrudniają analizę. Zbyt ogólne bloki zmniejszają możliwość wykrycia bloków ponownego użycia. Struktura nie może być zbyt duża i złożona.
- Staraj się wyizolować bloki ponownego użycia. Przeanalizuj podobieństwo nazw przypadków użycia, podobieństwo nazw i zachowania bloków ponownego użycia występujących w ich specyfikacji. Wydzielenie bloku ponownego użycia może być powiązane z określeniem bardziej ogólnej funkcji lub dodaniu nowej specjalizacji do istniejącej funkcji.
- Unikaj rozbicia jednego przypadku użycia na zbyt wiele podprzypadków.

1.8 Dokumentacja przypadków użycia

Diagramy przypadków użycia przedstawiają bardzo ogólny obraz systemu i nie pozwalają na przedstawienie wielu informacji, niezbędnych na potrzeby dalszych faz budowy systemu. Dlatego konieczne jest uzupełnienie diagramów użycia o dodatkową dokumentację. Dokument taki najczęściej składa się z następujących części:

- Nazwa – nazwa przypadku użycia, opisująca cel, który osiąga aktor w wyniku wykonania przypadku użycia. Zawsze zaczyna się od czasownika, np. „Zaloguj się do systemu” (niepoprawna nazwa to np. „Logowanie” lub „Obsługa logowania”).
- Aktor/aktorzy – wszyscy uczestnicy przypadku użycia (użytkownicy, zewnętrzne systemy lub urządzenie korzystające z przypadku użycia). Najpierw wymienia się głównego aktora, następnie aktorów drugoplanowych i systemy zewnętrzne. Nie należy wymieniać tu osób (aktorów) które tylko obserwują lub naprawiają błędy po fakcie.
- Zdarzenie inicjujące – konkretna akcja lub sytuacja, która uruchamia przypadek użycia, np. "Klient kliknie przycisk 'Prześlij pieniądze' w interfejsie aplikacji mobilnej". Inne możliwe triggerzy:
 - Zdarzenie czasowe ("Na koniec miesiąca system...")
 - Zmiana danych ("Gdy saldo spadnie poniżej...")
 - Żądanie zewnętrzne ("Gdy przychodzi żądanie z innego banku...")Częstym błędem jest opisywanie tu intencji użytkownika, a nie wykonanie przez niego konkretnej akcji, np. „Użytkownik chce się zalogować”. Chcenie nie jest zdarzeniem. Poprawne jest użycie w takiej sytuacji sformułowania „Gość kliknie przycisk *Zaloguj się*”.
- Warunki wstępne – warunki, które muszą być spełnione przed rozpoczęciem wykonywania przypadku użycia, opisujące stan systemu i otoczenia umożliwiający rozpoczęcie przypadku użycia, np.
 - Klient jest zalogowany w systemie
 - Konto klienta jest aktywne (nie zablokowane)
 - Klient ma wystarczające środki na koncie
 - Zapewniony jest dostęp do InternetuNiepoprawne jest odnoszenie się do wiedzy użytkownika, np. „Użytkownik zna swoje hasło”. Poprawnie określony warunek wstępny w takiej sytuacji to „Konto użytkownika istnieje i nie jest zablokowane”.
- Opis przebiegu interakcji – poszczególne kroki wykonania przypadku użycia – sekwencja akcji wyzwalanych przez aktora i reakcji systemu w odpowiedzi na nie. Opisz co robi system, a nie jak to robi.
- Warunki końcowe – efekty wykonania przypadku użycia, dające się zaobserwować w interfejsie systemu lub w jego interakcji z otoczeniem, opisujące stan systemu po pomyślnym zakończeniu przypadku użycia. Ważne: Warunki końcowe dotyczą tylko scenariusza sukcesu!
Przykład:
 - Pieniądze zostały przetransferowane z konta źródłowego
 - Konto odbiorcy zostało zasilone
 - Transakcja jest zarejestrowana w historii
 - Klient otrzymał potwierdzenie (SMS/email)
 - Saldo widoczne na koncie zostało zaktualizowane
- Alternatywny przepływ zdarzeń – wszystkie inne prawidłowe scenariusze, które różnią się od przepływu głównego, ale nadal **prowadzą do sukcesu** (tego samego celu, który jest wskazany w warunkach końcowych, osiągnąć w sposób opisany w przebiegu interakcji). Należy

rozdzielić alternatywny przepływ zdarzeń od sytuacji wyjątkowych – te drugie nie kończą się sukcesem.

- Sytuacje wyjątkowe – sytuacje, które powodują, że wykonanie przypadku użycia z sukcesem jest niemożliwe, ze wskazaniem warunków i spodziewanej reakcji systemu na tę sytuację. Dla każdego wyjątku należy tu opisać, jak system ma zareagować.

Porada praktyczna

Dobrze udokumentowany przypadek użycia powinien odpowiedzieć na pytania:

- KTO to robi? (Aktorzy)
- CO chce osiągnąć? (Nazwa)
- KIEDY się zaczyna? (Zdarzenie inicjujące)
- W JAKIM STANIE system musi być? (Warunki wstępne)
- JAK to wygląda krok po kroku? (Główny przepływ)
- CO się zmienia po zakończeniu? (Warunki końcowe)
- CO może pójść źle? (Sytuacje wyjątkowe)

Przykładowy opis przypadku użycia, w postaci tabeli, przedstawiony jest na **Rys. 11**.

Nazwa	Przesłanie pieniędzy na konto
Aktorzy	- Główny: Klient banku - Drugorzędni: System bankowy, Bank odbiorcy
Zdarzenie inicjujące	Klient klika przycisk "Prześlij pieniądze" w aplikacji
Warunki wstępne	1. Klient jest zalogowany 2. Konto jest aktywne 3. Dostępne są środki na koncie
Przebieg interakcji	1. Klient wybiera opcję "Prześlij pieniądze" 2. System wyświetla formularz przelewu 3. Klient wpisuje numer konta odbiorcy 4. System waliduje numer konta 5. Klient wpisuje kwotę i opis 6. Klient klika "Potwierdź" 7. System żąda kodu SMS 8. Klient wpisuje kod SMS 9. System weryfikuje kod i przetwarza przelew 10. System wyświetla potwierdzenie
Warunki końcowe	1. Pieniądze przetransferowane 2. Historia transakcji zaktualizowana 3. Potwierdzenie wysłane do klienta
Alternatywne przepływy	7a. Przesył międzynarodowy (dane SWIFT) 8a. Biometryczna weryfikacja
Sytuacje wyjątkowe	4a. Numer konta nieprawidłowy 5a. Niewystarczające środki 9a. Brak połączenia z serwerem

Rys. 11. Przykładowy opis przypadku użycia

2 Identyfikacja procesów biznesowych na podstawie historii użytkowników

2.1 Wstęp

Historia użytkownika ma postać:

jako – osoba, przypisana rola,
chcę – cecha, funkcjonalność, czynność,
ponieważ – uzasadnienie, rezultat, korzyść

Historyjki powinny być pogrupowane. Rozpatrzmy następujący przykład [3]:

Video Training Website

Poniżej zamieszczone są historyjki użytkowników, przygotowane na potrzeby pierwszej wersji strony internetowej, oferującej kursy wideo, udostępniane online przez różne prowadzące szkolenia online

Course Catalog

- As an instructor, I can create a description of a course so that prospective participants can learn about it.
- As an instructor, I can maintain a bio about myself so that it is used on any course listing for which I'm the instructor.
- As a site visitor, I can view a detailed page about each course so that I can learn all about it.
- As a site visitor, I can browse a catalog of all courses available so that I can pick the right one for me.
- As a site visitor, I can see a list of other related courses when viewing a course so that I can see other courses I might be interested in.

Account Management

- As a site visitor, I can create an account so that I can track my progress.
- As a participant, I can manage my account so that it reflects my current information.
- As a participant, I can view a list of all courses I've taken and the PDUs each earned me so that I have access to that historical information and don't call someone instead.

Course Creation

....

2.2 Identyfikacja ról i procesów biznesowych

Wstępna identyfikacja ról i procesów biznesowych polega na uporządkowaniu informacji zawartych w historyjkach użytkowników, w szczególności na wyodrębnieniu z nich ról (aktorów) i procesów biznesowych (przypadków użycia). Najwygodniej jest to przedstawić w postaci tabeli. Tabelę otrzymaną dla przykładu podanego we wstępie (zob. str. 13) pokazano poniżej (Tab. 1).

Tab. 1. Role i procesy biznesowe wyodrębnione z historyjek użytkownika dla systemu obsługi kursów

Rola biznesowa (aktor)	Proces biznesowy (przypadek użycia)	Uwagi
Course Catalog		

As an instructor , I can create a description of a course so that prospective participants can learn about it		
instructor	create a description of a course	
prospective participants	learn about a course	
As an instructor , I can maintain a bio about myself so that it is used on any course listing for which I'm the instructor.		
instructor	maintain a bio about myself	
unspecified	View the bio of an instructor who create a course	Co to za rola unspecified? Kto może oglądać profil instruktora? Jak to się ma do RODO?
As a site visitor, I can view a detailed page about each course so that I can learn all about it.		
site visitor	view a detailed page about each course	
As a site visitor, I can browse a catalog of all courses available so that I can pick the right one for me.		
site visitor	browse a catalog of all courses	
site visitor	pick the right course	Skąd visitor ma wiedzieć, że kurs jest dla niego odpowiedni? Jakie są kryteria „odpowiedniości”?
As a site visitor, I can see a list of other related courses when viewing a course so that I can see other courses I might be interested in.		
site visitor	see a list of other related courses when viewing a course	W sekcji Course Creation nie ma mowy o budowie zależności między kursami (related courses). Usunąć to wymaganie, albo uzupełnić Course Creation
Account Management		
site visitor	create an account	Czy site visitor to ta sama rola, co prospective participants, unspecified?
participant	manage my account	
participant	view a list of all courses I've taken and the PDUs each earned me	
participant	have access to historical information about all courses	Czy ta funkcja nie dubluje tej wyżej?

	I've taken and the PDUs each earned me	
--	--	--

Założmy, że uzyskaliśmy wyjaśnienia wątpliwości, uzyskując w efekcie następujący rezultat, stanowiący zatwierdzoną identyfikację ról i procesów biznesowych (Tab. 2).

Tab. 2. Zatwierdzona identyfikacja ról i procesów biznesowych dla systemu obsługi kursów

Rola biznesowa (aktor)	Proces biznesowy (przypadek użycia)
Course Catalog	
instructor	create a description of a course
site visitor	learn about a course
instructor	maintain a bio about myself
site visitor	view the bio of an instructor who create a course
site visitor	view a detailed page about each course
site visitor	browse a catalog of all courses
site visitor	pick the right course
site visitor	see a list of other related courses when viewing a course
Account Management	
site visitor	create an account
participant	track my progress
participant	manage my account
participant	view a list of all courses I've taken and the PDUs each earned me

Jak widać, wyodrębnione w efekcie role to:

1. Instructor
2. Site visitor
3. Participant

Uzyskane procesy biznesowe (przypadki użycia) są następujące:

1. instructor
 - 1.1. create a description of a course
 - 1.2. maintain a bio about myself
2. site visitor
 - 2.1. learn about a course (view a detailed page about each course)
 - 2.2. view the bio of an instructor who create a course
 - 2.3. browse a catalog of all courses
 - 2.4. pick the right course
 - 2.5. see a list of other related courses when viewing a course
 - 2.6. create an account
3. participant

- 3.1. track my progress
- 3.2. manage my account
- 3.3. view a list of all courses I've taken and the PDUs each earned me

2.3 Zakres usług systemu

Procesy otrzymane w p. 2.2 można następnie rozdzielić pomiędzy moduły systemu. Powstanie w ten sposób następujący zakres usług systemu:

1. Moduł katalogu kursów
 - 1.1. instructor
 - 1.1.1. create a description of a course
 - 1.1.2. maintain a bio about myself
 - 1.2. site visitor
 - 1.2.1. view a detailed page about a course
 - 1.2.2. view the bio of an instructor who create a course
 - 1.2.3. browse a catalog of all courses
 - 1.2.4. pick the right course
 - 1.2.5. see a list of other related courses when viewing a course
2. Moduł zarządzania kontem
 - 2.1. site visitor
 - 2.1.1. create an account
 - 2.2. participant
 - 2.2.1. track my progress
 - 2.2.2. manage my account
 - 2.2.3. view a list of all courses I've taken and the PDUs each earned me

2.4 Macierz odpowiedzialności

Specyfikację uprawnień do funkcji systemu można przedstawić w postaci prostej tabeli. Taka reprezentacja dla rozważanego przykładu zawarta jest w Tab. 3.

Tab. 3. Specyfikacja uprawnień do funkcji systemu obsługi kursów

PROCES BIZNESOWY	ROLA		
	INSTRUCTOR	SITE VISITOR	PARTICIPANT
Moduł katalogu kursów			
create a description of a course	X		
maintain a bio about myself	X		
view a detailed page about a course		X	
View the bio of an instructor who create a course		X	
browse a catalog of all courses		X	
pick the right course		X	
see a list of other related courses when viewing a course		X	
Moduł zarządzania kontem			
create an account		X	

track my progress			X
manage my account			X
view a list of all courses I've taken and the PDUs each earned me			X

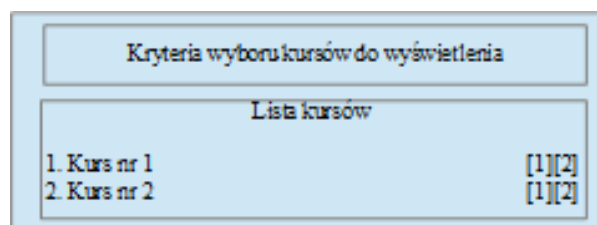
Jest to tzw. **macierz odpowiedzialności**, czyli macierz opisująca odpowiedzialność członków zespołu za realizację zadań projektu lub procesu biznesowego. Macierz ta znana jest również jako macierz RAM – Responsibility Assignment Matrix lub macierz RACI. RACI jest akronimem wywodzącym się z czterech kluczowych obowiązków występujących podczas realizacji projektu: Responsible, Accountable, Consulted, Informed.

2.5 Makiety GUI

W metodzie User Centered Design istotne jest szybkie dostarczenie **pretotypu**¹ produktu, po to żeby użytkownik mógł jak najszybciej ocenić jego funkcjonalność. W przypadku oprogramowania z interfejsem użytkownika pretotypem są makiety interfejsu użytkownika.

Makiety można tworzyć przy pomocy dedykowanego oprogramowania albo naszkicować na kartce.

Dla naszego przykładu (zob. 2.1) makieta procesu o nazwie *Browse a catalog of all courses* mogłyby wyglądać tak, jak pokazano na Rys. 12.

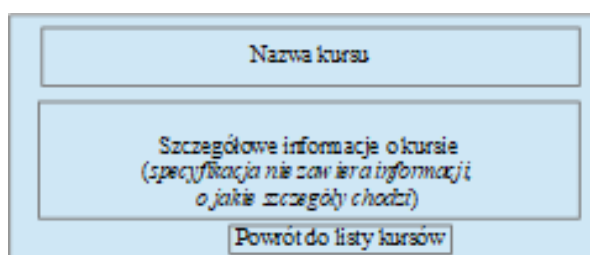


Rys. 12. Makieta procesu o nazwie *Browse a catalog of all courses*

Wybranie symbolu „[1]” powoduje wyświetlenie szczegółów kursu (proces *View a detailed page about a course*).

Wybranie symbolu „[2]” powoduje wyświetlenie informacji o autorze kursu (proces *View the bio of an instructor who create a course*).

Przykładową makietę procesu *View a detailed page about a course* pokazano na Rys. 13.



Rys. 13. Makieta procesu *View a detailed page about a course*

¹ Pretotyp to coś innego niż prototyp. Sprawdź notatki z wykładu lub zaglądaj do Internetu, np. tu: <https://marketerplus.pl/pretotyping-ograniczyc-straty-uniknac-fiaska-nowych-produktachpomyslachuslugach/>.

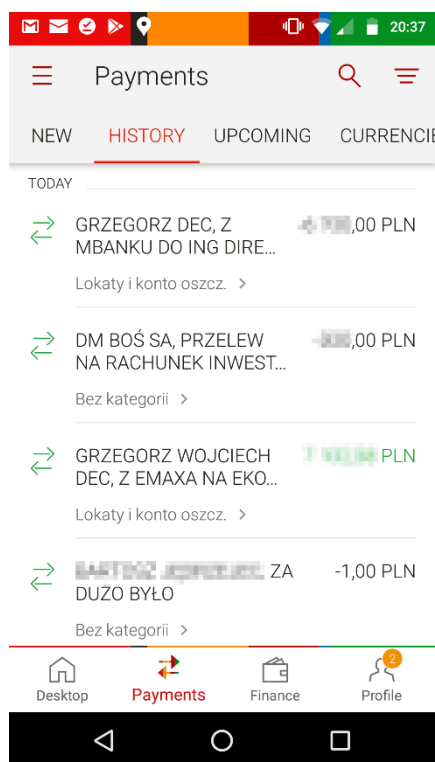
3 Identyfikacja struktur danych na podstawie historii użytkowników

3.1 Wstęp

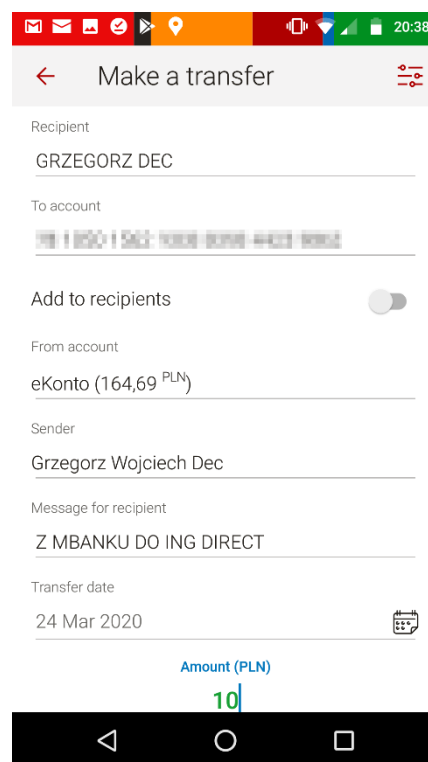
W rozdziale przedstawiono zagadnienia związane z analizą procesów biznesowych pod kątem identyfikacji struktur danych, potrzebnych do zdefiniowania w systemie informatycznym.

Zostaną one pokazane na przykładzie aplikacji dla banku. Załóżmy, że mamy następujące historyjki użytkowników:

1. Jako zalogowany użytkownik chcę widzieć historię operacji wybranego rachunku, ponieważ *[nie wiem co napisać, bo jestem informatykiem, a nie marketingowcem]*.
2. Jako zalogowany użytkownik chcę wykonać przelew z wybranego rachunku na inny rachunek, ponieważ *[nie wiem co napisać, bo jestem informatykiem, a nie marketingowcem]*.



Rys. 14. Przykładowy raport



Rys. 15. Przykładowy formularz

Historyjkę 1 implementuje funkcja pokazana na Rys. 14. Na Rys. 15 pokazano implementację historyjki 2.

Okno z Rys. 14 wyświetla strukturę danych o nazwie historia operacji rachunku. Aby mogły się w nim pojawić informacje o historii operacji na koncie użytkownika, system musi mieć zaimplementowaną odpowiednią funkcjonalność, np. w postaci następującej funkcji:

```
HistoriaOperacjiRachunku pobierzHistorieOperacji(Rachunek r);
```

Czym jest `HistoriaOperacjiRachunku`? Patrząc na Rys. 14 widzimy, że jest to rodzaj kolekcji, która zawiera informacje o kilku operacjach. W języku C++ napisalibyśmy:

```
class HistoriaOperacjiRachunku : public std::set<Operacja> { ... }
```

Ponieważ funkcja biznesowa *Prezentuj historię operacji rachunku* prezentuje dane, powiemy że jest to **raport**.

Na Rys. 15 mamy funkcję biznesową *Wykonaj przelew*. System zapewne będzie miał taką funkcję:

```
void wykonajPrzelew(Przelew p);
```

Funkcja ta nie prezentuje żadnych danych, tylko zapisuje w bazie danych informacje o przeprowadzonej operacji. Obiekt *p* klasy *Przelew* jest tworzony na podstawie danych wprowadzonych przez użytkownika systemu. Takie funkcje nazywamy **formularzami**.

Przeanalizujmy dane występujące w historyjkach:

1. Jako **zalogowany użytkownik** chcę widzieć **historię operacji wybranego rachunku**, ponieważ *nie wiem co napisać, bo jestem informatykiem, a nie marketingowcem*.
2. Jako **zalogowany użytkownik** chcę wykonać **przelew z wybranego rachunku** na inny **rachunek**, ponieważ *nie wiem co napisać, bo jestem informatykiem, a nie marketingowcem*.

Z wcześniejszego opisu wiemy, że *HistoriaOperacjiRachunku* to kolekcja obiektów klasy *Operacja*, tworzona przez funkcję *pobierzHistorieOperacji*. Nas interesują takie struktury danych, które nie są tworzone przez funkcje. Szukamy takich danych, które są pobierane albo zapisywane w tabelach w bazie danych. Takimi strukturami w przykładzie z bankiem mogą być:

1. Operacja
2. Rachunek
3. Przelew
4. Użytkownik

Widzimy, że w systemie są dwie klasy danych:

1. Obiekty transferowe² (w programowaniu obiektowym są to obiekty, które przechowują tylko pola publiczne, bez metod) - w naszym przykładzie jest to *HistoriaOperacjiRachunku*.
2. Komponenty encyjne (te wymienione wyżej).

Obiekty transferowe i komponenty encyjne występują zwykle w historyjkach użytkowników, ale są wymieszane ze sobą. Musimy się nauczyć znajdować komponenty encyjne.

Komponent encyjny to implementacja encji w wybranym języku programowania. Co to jest encja?

Encja to abstrakcja opisująca dane przechowywane przez system.

W programowaniu obiektowym taką abstrakcją jest **klasa**, jako opis abstrakcyjny zbioru obiektów o identycznych właściwościach i zachowaniu. Na podstawie klasy tworzymy obiekty.

Jej odpowiednikiem w analizie systemów informatycznych jest **encja**. Pojęciem tym określa się opis abstrakcyjny zbioru obiektów o identycznych właściwościach. Na podstawie encji tworzymy tabele w bazie danych. Na podstawie encji można też tworzyć klasy w językach programowania obiektowego.

Między encjami występują związki. W przykładzie z bankiem związki te mogą być następujące:

1. Każdy **użytkownik** posiada wiele **rachunków**.
2. Każdy rachunek ma wiele **operacji**.
3. **Przelew** jest wykonany z **rachunku**.

² <https://docplayer.pl/6599496-Wzorce-projektowe-warstwy-danych.html>

4. **Przelew** jest wykonany na **rachunku**.
5. **Użytkownik** może wykonać wiele **przelewów**.

3.2 Analiza danych z historyjek użytkowników

Analizę danych z historyjek użytkowników przeprowadzimy na przykładzie z rozdziału 2.1 [3]:

Course Catalog

- As an instructor, I can create a description of a course so that prospective participants can learn about it.
- As an instructor, I can maintain a bio about myself so that it is used on any course listing for which I'm the instructor.
- As a site visitor, I can view a detailed page about each course so that I can learn all about it.
- As a site visitor, I can browse a catalog of all courses available so that I can pick the right one for me.
- As a site visitor, I can see a list of other related courses when viewing a course so that I can see other courses I might be interested in.

Account Management

- As a site visitor, I can create an account so that I can track my progress.
- As a participant, I can manage my account so that it reflects my current information.
- As a participant, I can view a list of all courses I've taken and the PDUs each earned me so that I have access to that historical information and don't call someone instead.

Course Creation

- As an instructor, I can upload videos to be part of a course so that participants can watch those videos.
- As an instructor, I can create a quiz to include in a video course so that participants can take those quizzes to ensure they are learning.
- As an instructor, I can properly sequence the elements of course so that I can assemble them into an amazing course.
- As an instructor, I indicate whether a discussion forum is part of a course so that I can add a forum to some but not all courses.
- As an instructor, I can indicate the schedule for a course so that prospective participants and participants can see what course elements will be presented each week and what they'll be.
- As an instructor, I can set the publication date for course elements (for a scheduled course) so that items become available when I want and not sooner.
- As an instructor, I can add subtitles to any video so that it is more accessible.
- As an instructor, I can set the thumbnail image for each video so that the image is one I like rather than randomly selected.
- As a site owner, I want all videos to be watermarked with a Front Row Agile logo so that the videos are clearly from the Front Row Agile site.

3.2.1 Wstępna identyfikacja struktur danych

Wstępnej identyfikacji struktur danych można dokonać w oparciu o przegląd zidentyfikowanych poprzednio procesów biznesowych, zaznaczając w nim struktury danych, na których procesy

przeprowadzają operacje. Dla naszego przykładu można zidentyfikować w ten sposób struktury wyróżnione w Tab. 4 tekstem pogrubionym.

Tab. 4. Wstępnie zidentyfikowane struktury danych dla systemu obsługi kursów

Rola biznesowa (aktor)	Proces biznesowy (przypadek użycia)
Course Catalog	
instructor	create a description of a course
site visitor	learn about a course
instructor	maintain a bio about myself
site visitor	view the bio of an instructor who create a course
site visitor	view a detailed page about each course
site visitor	browse a catalog of all courses
site visitor	pick the right course
site visitor	see a list of other related courses when viewing a course
Account Management	
site visitor	create an account
participant	track my progress
participant	manage my account
participant	view a list of all courses I've taken and the PDUs each earned me
Course Creation	
instructor	upload videos to be part of a course
participant	watch a video that is a part of a course
instructor	create a quiz to include in a video course
participant	take a quiz to ensure the participant is learning
instructor	sequence the elements of a course
instructor	add a forum to a course
instructor	indicate the schedule for a course
prospective participant	see what course elements will be presented each week and what they'll be
instructor	set the publication date for course elements
system	course elements become available when scheduled
instructor	add subtitles to a video
instructor	set the thumbnail image for a video
site owner	watermark all videos with a logo

3.2.2 Identyfikacja duplikatów i zdefiniowanie struktur danych

Struktury danych zidentyfikowane wstępnie z procesów biznesowych należy następnie poddać analizie, pod kątem określenia, które z nich opisują te same dane. System należy tu traktować jako całość, bez dzielenia go na moduły. W efekcie otrzymamy na ogół pary: [struktura, lista struktur].

Tab. 5. Zbiór struktur danych zidentyfikowanych dla systemu obsługi kursów

Nazwa struktury	Zidentyfikowana struktura danych
course	description of a course
course	a course
instructor	a bio
instructor	the bio of an instructor
course	a detailed page about each course
courses_list	a catalog of all courses
course	the right course
courses_list	a list of other related courses
participant	an account
courses_list	participant's progress
participant	participants account
courses_list	a list of all participant's courses
video_list	videos (a parts of a course)
video	a video
course	a course
video	a video course
quiz	a quiz (for a video course)
video	the elements of a course
forum	a forum to a course
schedule	the schedule for a course
video	course elements
schedule_item	the publication date for course elements
video	course elements
subtitles	subtitles to a video
thumbnail_image	the thumbnail image for a video
logo	a logo for a video

3.2.3 Klasyfikacja procesów biznesowych

Z punktu widzenia użytkownika systemu, procesy biznesowe dzielimy na 3 typy:

1. Formularze do wprowadzania, aktualizacji lub usuwania danych (F)
2. Raporty prezentujące dane (R)
3. Procesy obliczeniowe, które mogą utworzyć lub zaktualizować dane (P)

Klasyfikacja taka pomaga odróżnić te struktury danych, które są przechowywane w bazie danych (encje), od tych, które służą do prezentacji. Raport może powstać na podstawie danych z kilku tabel (obiekt transferowy).

3.2.4 Przypisanie struktur danych do procesów biznesowych

W kolejnym kroku w procesach biznesowych, służących do wstępnej identyfikacji struktur danych (zob. p. 3.2.1), dla każdej funkcji biznesowej należy określić jej typ, zgodnie z klasyfikacją wskazaną w p. 3.2.3. Dla przykładu obsługi kursów otrzymamy w efekcie Tab. 6.

Tab. 6. Typy funkcji biznesowych w systemie obsługi kursów

Nr	Rola biznesowa (aktor)	Proces biznesowy (przypadek użycia)	Typ
Course Catalog			
1	instructor	create a description of a course	F
2	site visitor	learn about a course	R
3	instructor	maintain a bio about myself	FR
4	site visitor	view the bio of an instructor who create a course	R
5	site visitor	view a detailed page about each course	R
6	site visitor	browse a catalog of all courses	R
7	site visitor	pick the right course	F
8	site visitor	see a list of other related courses when viewing a course	R
Account Management			
9	site visitor	create an account	F
10	participant	track my progress	R
11	participant	manage my account	FR
12	participant	view a list of all courses I've taken and the PDUs each earned me	R
Course Ceation			
13	instructor	upload videos to be part of a course	F
14	participant	watch a video that is a part of a course	R
15	instructor	create a quiz to include in a video course	F
16	participant	take a quiz to ensure the participant is learning	FR
17	instructor	sequence the elements of a course	F

18	instructor	add a forum to a course	F
19	instructor	indicate the schedule for a course	F
20	prospective participant	see what course elements will be presented each week and what they'll be	R
21	instructor	set the publication date for course elements	F
22	system	course elements become available when scheduled	P
23	instructor	add subtitles to a video	F
24	instructor	set the thumbnail image for a video	F
25	site owner	watermark all videos with a logo	F

3.2.5 Identyfikacja encji

Na potrzeby powiązania wyodrębnionych wcześniej struktur danych z encjami lub z obiektami transferowymi, należy rozdzielić je na te, które służą wyłącznie do przesyłania danych do raportów i mogą być zbudowane przez funkcje, oraz te, które powinny być zapisywane w bazie danych.

Dla przykładu obsługi systemu kursów otrzymamy encje zebrane w Tab. 7.

Tab. 7. Encje wyodrębnione dla systemu obsługi kursów

Nr	Nazwa
	Instructor
	Course
	Thumbnail image
	Schedule
	Forum
	Participant
	Video
	Subtitle
	Logo
	Schedule_item
	Quiz

3.2.6 Klasyfikacja procesów biznesowych pod kątem operacji bazodanowych

Relacyjna baza danych przeprowadza 4 typy operacji:

1. **Create** (wstawianie danych do tabel)
2. **Retrieve** (pobieranie danych)
3. **Update** (aktualizacja danych)
4. **Delete** (usuwanie danych)

Aby określić, z jakim rodzajem encji mamy do czynienia, dobrze jest opisać relacje łączące funkcje biznesowe z p. 3.2.4 z encjami i typem operacji. Najlepiej jest to zrobić w postaci tabeli, opartej na zidentyfikowanych wcześniej procesach biznesowych (zob. p. 3.2.4) i encjach (zob. p. 3.2.5).

Umieszczając procesy biznesowe w wierszach tabeli, a encje w kolumnach, w komórkach odpowiadających danej kombinacji *proces biznesowy* – *encja* można wskazać typy operacji dla

relacyjnej bazy danych, odpowiadające takim kombinacjom. Zestawienie operacji bazodanowych dla procesów biznesowych i struktur danych dla rozważanego w rozdziale systemu obsługi kursów, zwane **macierzą CRUD**, pokazana jest w Tab. 8.

Tab. 8. Macierz CRUD dla systemu obsługi kursów.

	instructor	course	thumbnail image	schedule	forum	participant	video	subtitle	Logo	schedule item	quiz
create a course		C									
Learn about a course		R									
maintain an instructor (my bio)	CRUD										
View the instructor (who create a course)	R										
view a course		R									
browse courses_list		R									
pick the course											
see a courses_list (related courses when viewing a course)		R									
create a participant						C					
track my progress (view my past courses_list)											
manage participant (my account)						CRUD					
view courses_list (a list of all courses I've taken)											
upload video_list to be part of a course							C				
watch a video that is a part of a course							R				
create a quiz to include in a course											C
take a quiz to ensure the participant is learning											R
Sequence video_list (the elements of a course)							U				
add a forum to a course					C						
indicate the schedule for a course				C							
see the schedule for a course				R							
set the schedule_item (publication date for a video)										C	
video_list become available when scheduled											
add subtitles to a video								C			
set the thumbnail image for a video			C								
watermark video_list with a logo							U		C		

Zasady konstrukcji tabeli są następujące:

1. Jeżeli funkcja biznesowa tworzy instancje encji w bazie danych (czyli zapisuje dane do tabeli), to na przecięciu z tą encją piszę **C**.
2. Jeżeli funkcja biznesowa pobiera instancje encji z bazy danych (czyli odczytuje dane z tabeli), to na przecięciu z tą encją piszę **R**.
3. Jeżeli funkcja biznesowa aktualizuje instancje encji, to na przecięciu z tą encją piszę **U**.
4. Jeżeli funkcja biznesowa usuwa instancje encji, to na przecięciu z tą encją piszę **D**.
5. Funkcje mogą być złożone, wtedy piszemy kilka liter.

W Tab. 8 kolorem zaznaczone są te funkcje biznesowe, dla których nie udało się przypisać encji, przy czym:

- Funkcja *video_list become available when scheduled* jest funkcją wewnętrzną, która nie pozwala wyświetlić video poza harmonogramem.

- Funkcje *pick the course*, *track my progress (view my past courses_list)*, *view courses_list (a list of all courses I've taken)* na pierwszy rzut oka działają na encji *course*, ale sprawa jest trochę bardziej złożona.

Rozpatrzmy przykład. W bibliotece mamy książki. Encja **książka** opisuje dane książki i jest abstrakcją. Książki nie da się przeczytać. Na półce może stać kilka **egzemplarzy danej książki**. Egzemplarz książki nazywa się **wolumin**. Ten wolumin da się czytać i można go wypożyczyć. Gdy wypożyczam książkę, trzeba to zarejestrować. Wprowadzamy encję **wypożyczenie**. O bibliotece powiemy:

1. **Czytelnik** posiada wiele **wypożyczeń**.
2. **Wypożyczenie** dotyczy jednego **woluminu książki**.
3. **Książka** opisuje wiele **woluminów książek**.

W rozważanym w rozdziale przykładzie dotyczącym systemu obsługi kursów jest podobnie:

1. **Instruktor** tworzy wiele **kursów**.
2. **Kurs** ma **harmonogram**. **Harmonogram** ma wiele pozycji (**schedule_items**).
3. **Uczeń** deklaruje **przystąpienie do kursu**. **Przystąpienie do kursu** wiąże ucznia z **harmonogramem**.

Mówiąc inaczej: **kurs** jest obiektem, który licencjonujemy uczniom. Trzeba zapisać, że dany uczeń przystąpił do kursu. To **przystąpienie do kursu** jest encją, na której operują funkcje *pick the course*, *track my progress (view my past courses_list)*, *view courses_list (a list of all courses I've taken)*.

Analogiczna sytuacja jest z encją **quiz** i funkcją *take a quiz to ensure the participant is learning*.

3.2.7 Korekta zbioru encji

W przypadku wystąpienia sytuacji podobnych do wskazanych w punkcie 3.2.6 należy wprowadzić nowe encje, a następnie uaktualnić arkusz CRUD. Dla rozważanego w rozdziale systemu obsługi kursów powinny zostać wprowadzone dodatkowo encje **Participants course** i **Participants quiz**, pozwalające na przechowanie informacji o tym, że dany uczeń przystąpił do kursu (quizu). Wobec tego zaktualizowana została Tab. 7. Zestaw encji uwzględniający dodatkowo encje **Participants course** i **Participants quiz** pokazany jest w Tab. 9.

Tab. 9. Encje wyodrębnione dla systemu obsługi kursów po korekcie

Nr	Nazwa
	Instructor
	Course
	Thumbnail image
	Schedule
	Forum
	Participant
	Video
	Subtitle
	Logo
	Schedule_item
	Quiz
	Participants course
	Participants quiz

Uaktualnioną w efekcie tych zmian macierz CRUD z Tab. 8 pokazano w Tab. 10.

Tab. 10. Macierz CRUD dla systemu obsługi kursów po korekcie uwzględniającej dodane encje

	instructor	course	thumbnail image	schedule	forum	participant	video	subtitle	Logo	schedule item	quiz	part. Course	part. Quiz
create a course		C											
Learn about a course		R											
maintain an instructor (my bio)	CRUD												
View the instructor (who create a course)	R												
view a course		R											
browse courses_list		R											
pick the course												C	
see a courses_list (related courses when viewing a course)		R											
create a participant						C							
track my progress (view my past courses_list)												R	
manage participant (my account)						CRUD							
view courses_list (a list of all courses I've taken)												R	
upload video_list to be part of a course							C						
watch a video that is a part of a course							R						
create a quiz to include in a course											C		
take a quiz to ensure the participant is learning											R		C
Sequence video_list (the elements of a course)							U						
add a forum to a course					C								
indicate the schedule for a course				C									
see the schedule for a course				R									
set the schedule_item (publication date for a video)										C			
video_list become available when scheduled													
add subtitles to a video								C					
set the thumbnail image for a video			C										
watermark video_list with a logo							U		C				

3.2.8 Elementarne funkcje biznesowe

Komórki macierzy CRUD, w których znajduje się kilka operacji bazodanowych (kilka liter), tak jak przy funkcji *maintain an instructor (my bio)* w Tab. 10, zwykle związane są z funkcjami (procesami biznesowymi), które grupują prostsze funkcje. W przytoczonym przykładzie z Tab. 10 funkcja *maintain an instructor (my bio)* składa się z czterech funkcji elementarnych:

1. Maintain an instructor (my bio).
 - 1.1. Create instructor's profile
 - 1.2. Show instructor's profile
 - 1.3. Update instructor's profile
 - 1.4. Delete instructor's profile

Podobnie jest z funkcją *manage participant (my account)*:

1. Manage participant (my account).
 - 1.1. Create participant's profile
 - 1.2. Show participant's profile
 - 1.3. Update participant's profile
 - 1.4. Delete participant's profile

Po rozłożeniu złożonych funkcji biznesowych na składowe elementarne otrzymujemy ostateczną macierz CRUD. Dla przykładu systemu obsługi kursów otrzymamy w efekcie macierz CRUD pokazaną w Tab. 11.

Tab. 11. Macierz CRUD dla systemu obsługi kursów po rozłożeniu funkcji biznesowych na składowe elementarne

	instructor	course	thumbnail image	schedule	forum	participant	video	subtitle	Logo	schedule item	quiz	part. Course	part. Quiz
create a course		C											
Learn about a course		R											
Create an instructor (my bio)	C												
Show an instructor (my bio)	R												
Update an instructor (my bio)	U												
Delete an instructor (my bio)	D												
View the instructor (who create a course)	R												
view a course		R											
browse courses_list		R											
pick the course												C	
see a courses_list (related courses when viewing a course)		R											
track my progress (view my past courses_list)												R	
create a participant						C							
Show participant (my account)						R							
Update participant (my account)						U							
Delete participant (my account)						D							
view courses_list (a list of all courses I've taken)												R	
upload video_list to be part of a course							C						
watch a video that is a part of a course							R						
create a quiz to include in a course											C		
take a quiz to ensure the participant is learning											R		C
Sequence video_list (the elements of a course)							U						
add a forum to a course					C								
indicate the schedule for a course				C									
see the schedule for a course				R									
set the schedule_item (publication date for a video)										C			
video_list become available when scheduled													
add subtitles to a video								C					
set the thumbnail image for a video			C										
watermark video_list with a logo							U		C				

3.2.9 Kompletność modeli

Macierz CRUD pozwala uniknąć błędów na poziomie projektowania funkcjonalności systemu, poprzez umożliwienie weryfikacji kompletności modeli, polegającej na sprawdzeniu, czy dla każdej encji przewidziano wszystkie operacje bazodanowe (C, R, U i D).

Zestawienie otrzymane na podstawie Tab. 11 dla systemu obsługi kursów pokazano w Tab. 12. Na czerwono zaznaczono komórki odpowiadające brakującym operacjom bazodanowym dla

powiązanych z nimi encji. Niektóre z takich błędów są proste do usunięcia, ale niektóre wymagają dodatkowych prac analitycznych.

Tab. 12. Zestawienie operacji CRUD dla encji w systemie obsługi kursów

	instructor	course	thumbnail image	schedule	forum	participant	video	subtitle	Logo	schedule item	quiz	part. Course	part. Quiz
Create	1	1	1	1	1	1	1	1	1	1	1	1	1
Retrieve	2	4	0	1	0	1	1	0	0	0	1	2	0
Update	1	0	0	0	0	1	2	0	0	0	0	0	0
Delete	1	0	0	0	0	1	0	0	0	0	0	0	0
Error	YES	YES	YES	YES	YES	YES	YES	YES	YES	YES	YES	YES	YES

Widzimy, że encja **course** nie ma przewidzianych operacji UD. Rzeczywiście, historyjki użytkowników nie przewidują tego. Zwracamy się do zamawiającego system, żeby uzupełnił brak i wystawiamy mu fakturę za konsultacje projektu.

Encja **thumbnail image** nie ma operacji RUD. Obrazek podglądu jest jednak wyświetlany podczas przeglądania **video**. Szukamy funkcji, które mają R dla encji **video** i zastanawiamy się, czy dopisać w nich R dla encji thumbnail image. Znajdziemy funkcję *watch a video that is a part of a course*. Ale operacji UD już nie przewidziano. Obrazek podglądu zostaje przypisany do video na zawsze.

Jeżeli uzupełniamy model o nowe funkcje biznesowe, pamiętajmy o ponowieniu wyceny. Te nowe funkcje będą na ogół elementarnymi funkcjami biznesowymi o prostym stopniu skomplikowania.

4 Modelowanie danych za pomocą diagramu związków encji (ERD)

4.1 Wstęp

Faza implementacji wytwarzania oprogramowania jest procesem tworzenia wykonywalnego systemu, który może być dostarczony klientowi. Projekt oprogramowania jest opisem:

- struktury oprogramowania, które ma zostać zaimplementowane,
- modeli danych i struktur używanych przez system,
- interfejsów między poszczególnymi komponentami,
- a niekiedy również użytych algorytmów.

Większość oprogramowania współdziała z innymi systemami, takimi jak system operacyjny, baza danych, oprogramowanie pośredniczące i inne systemy użytkowe. Łącznie tworzą one „platformę oprogramowania”, czyli środowisko, w którym tworzone oprogramowanie będzie działać. Informacje o tej platformie stanowią podstawowe dane wejściowe procesu projektowania, gdyż projektanci muszą zdecydować, jak najlepiej zintegrować tworzone oprogramowanie z jego środowiskiem.

Jeśli system ma przetwarzać istniejące dane, opis tych danych może być dołączony do specyfikacji platformy. W przeciwnym razie opis danych musi być informacją wejściową dla procesu projektowania, aby możliwe było zdefiniowanie organizacji danych.

W procesie projektowania systemu, na potrzeby opisanego go w celu implementacji, a po implementacji także do dokumentowania struktury i działania systemu wykorzystuje się modele.

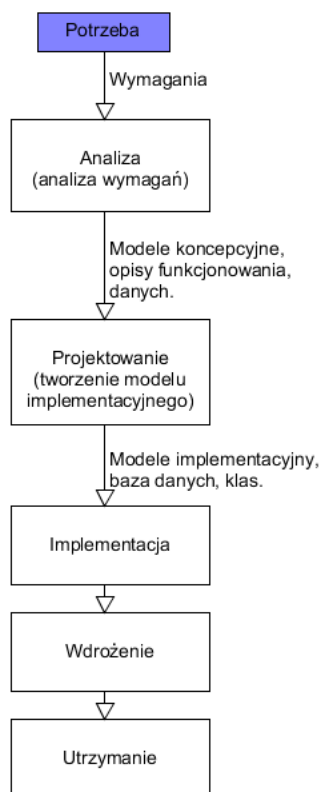
Różne modele pozwalają przedstawić system z różnych perspektyw, na przykład:

- z perspektywy zewnętrznej, gdy modelujemy kontekst lub środowisko systemu,
- z perspektywy interakcji, gdy modelujemy interakcje między systemem a jego środowiskiem lub między komponentami systemu,
- z perspektywy strukturalnej, gdy modelujemy organizację systemu lub strukturę danych przetwarzanych przez ten system,
- z perspektywy behawioralnej, w której modelujemy dynamiczne zachowanie systemu i to, jak odpowiada na zdarzenia.

Na potrzeby prezentacji danych z różnych perspektyw można wykorzystać następujące typy modeli danych:

- model koncepcyjny,
- model fizyczny,
- model implementacyjny.

Są one przydatne przy budowie systemu informatycznego, na etapie poprzedzającym jego implementację, wdrożenie i utrzymanie (zob. Rys. 16).



Rys. 16. Schemat procesu tworzenia systemu informatycznego

4.1.1 Model koncepcyjny

Model koncepcyjny pozwala na opisanie danych za pomocą pojęć, z których korzystają użytkownicy. Jest to model ogólny, wysokopoziomowy, całkowicie niezależny od tworzonego rozwiązania, a w szczególności struktury bazy danych, jaka będzie ewentualnie utworzona. Model ten prezentuje perspektywę biznesową. Określa obiekty istniejące w systemie lub domenie i sposób, w jaki są one ze sobą powiązane.

4.1.2 Model fizyczny

Model fizyczny opisuje dane w kategoriach sposobu ich przechowywania w pamięci komputera. Odzwierciedla on więc organizację bazy danych. Tworząc go należy uwzględnić zagadnienia techniczne, związane z normalizacją, wydajnością, bezpieczeństwem itp. Zwykle jest on przedmiotem zainteresowania specjalistów w zakresie baz danych, a więc deweloperów, architektów etc.

4.1.3 Model implementacyjny

Model implementacyjny stanowi grupę pośrednią, operując pojęciami wprawdzie zrozumiałymi dla użytkowników, ale bliżej związanymi z konkretną (logiczną) strukturą danych w pamięci komputera. Zawiera informacje pozwalające na implementację. Wiąże to co chce klient biznesowy z tym, jak te wymagania spełnić.

Obejmuje trzy podstawowe modele: relacyjny, sieciowy i hierarchiczny.

4.2 Model związków encji (ERM) i diagram związków encji (ERD)

Najpopularniejszym koncepcyjnym modelem danych jest tzw. **model związków encji (ERM – Entity Relationship Model)**. Jego graficznym odpowiednikiem jest **diagram związków encji (ERD – Entity Relationship Model)**.

Diagram ERD umożliwia modelowanie rzeczywistych obiektów świata na potrzeby tworzenia systemu informatycznego. Takie odwzorowanie (model) pozwala określić system pod kątem jego struktury oraz modeli danych. Podstawowe informacje do wykonania diagramu ERD pochodzą z potrzeby oraz wymagań (zob. Rys. 16).

Diagramy ERD pokazują obiekty, wyodrębnione na podstawie wymagań dla tworzonego systemu informatycznego. Obiekty te nazywane są encjami i są scharakteryzowane za pomocą cech, określanych mianem atrybutów. Diagramy opisują także relacje, jakie zachodzą między obiektami (encjami) [4]. Diagram ERD spotyka się w różnych graficznie postaciach, do których zaliczamy notacje Chena, Martina, Bachmana, IDEF1X i inne. Przykładowe diagramy ERD w różnych notacjach pokazano na Rys. 17.

Diagram ERD pozwala na zrozumienie struktury danych, przygotowanie późniejszej strategii optymalizacji bazy oraz stanowi podstawową dokumentację systemu przechowywania informacji. Jest częścią wielu popularnych narzędzi CASE (Computer Aided Software Engineering), pozwalających na wygenerowanie fizycznej struktury bazodanowej na podstawie zaprojektowanego diagramu. Systemy CASE, które wspierają tworzenia tych diagramów, mogą na ich podstawie automatycznie tworzyć bazy danych odpowiadające relacjom na diagramie.

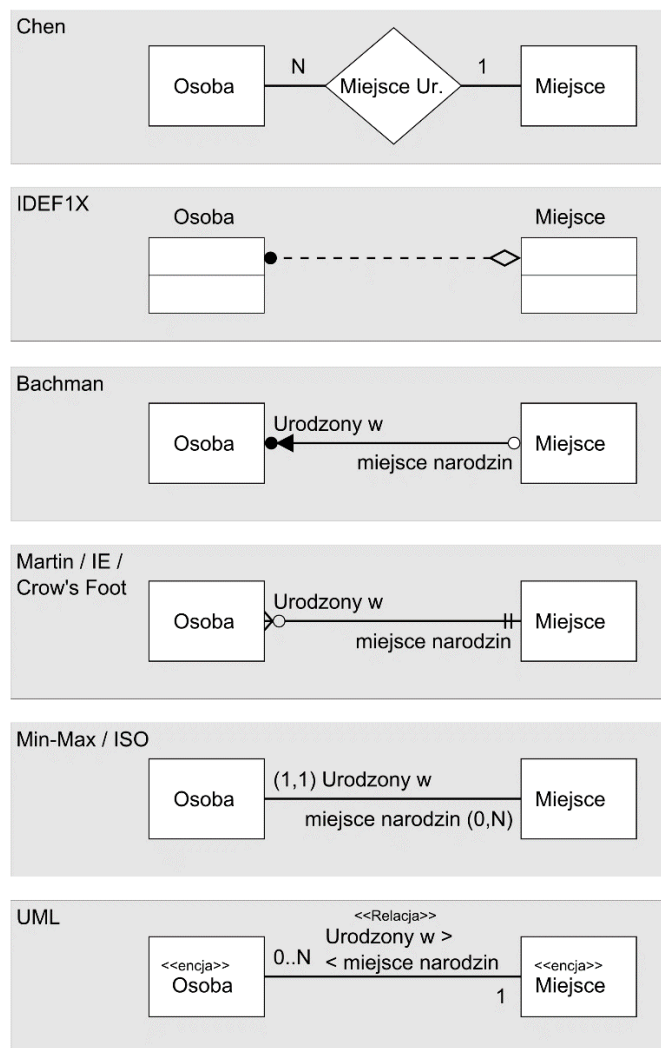
Relacje pomiędzy encjami opisywane są za pomocą następujących cech:

- Liczebność (krotność), oraz
- Opcjonalność.

Opcjonalność określa, czy każda encja **musi**, czy też **może** wystąpić równocześnie z inną. Np. TOWAR **musi** zostać zakupiony przez co najmniej jednego KLIENTA, ale KLIENT **może** być nabywcą TOWARU. W reprezentacji graficznej linia przerywana oznacza opcjonalność związku, natomiast ciągła wymóg związku.

Liczebność (krotność) z kolei określa, ile encji wchodzi w skład związku. Może one wystąpić w formie związku:

- 1:1 (jeden do jeden) – encji odpowiada dokładnie jedna encja,
- 1:N (jeden do wielu) – encji odpowiada jedna lub więcej encji,
- M:N (wiele do wielu) – jednej lub więcej encjom odpowiada jedna lub więcej encji.



Rys. 17. Przykładowe diagramy ERD w różnych notacjach

W przypadku związków M:N często stosuje się **normalizację diagramu**, która polega na dodaniu encji pośredniczącej i zastąpienie związku M:N dwoma związkami 1:N z nową encją.

4.2.1 Elementy modelu ERD

W skład modelu związków encji wchodzi elementy takie jak: **encja (obiekt)**, **atrybut**, **związek**. Obiekty mogą reprezentować byty rzeczywiste lub mogą być pojęciami abstrakcyjnymi. Obiekty mające te same atrybuty łączy się w typy obiektów np. Towar, Klient, Dostawca, Zamówienie.

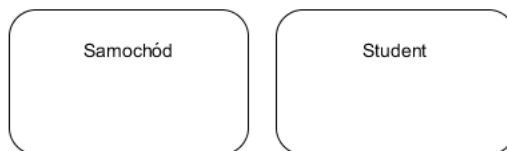
Encja

Encja (ang. entity) to reprezentacja zbioru obiektów opisanych identycznymi cechami (atrybutami). Encje wyrażają więc różnego rodzaju obiekty. Mogą nimi być składniki rzeczywistości, ale też szeroko pojęte byty - rzeczy lub obiekty mające dla nas znaczenie, rzeczywiste bądź wyobrażone, o którym informacje muszą być znane lub przechowywane. Przykładem encji mogą być np. samochód lub faktura. Na podstawie encji tworzone są struktury w bazie danych np. tabele.

Podstawowym elementem encji jest jej unikalna nazwa, opisana jako rzeczownik w liczbie pojedynczej np. samochód, faktura, student, a w przypadku magazynu danych w liczbie mnogiej. Do

stworzenia encji, poza nazwą, wymagane jest podanie przynajmniej jednego atrybutu (identyfikatora) encji. Nie ma jednak ograniczenia, co do wielkości zbioru atrybutów encji.

W diagramie encja prezentowana jest najczęściej w formie prostokąta o ostrych, bądź zaokrąglonych rogach. Nazwa danej encji umiejscowiona jest zazwyczaj w pierwszym wersie, a dodatkowo oznaczana jest innym kolorem, bądź też rodzajem czcionki – tak, aby łatwo było ją dostrzec. Przykładowe encje w diagramie ERD pokazane są na Rys. 18.



Rys. 18. Przykład prostych encji w diagramie ERD

Dwie podstawowe własności encji są następujące:

- obiekty wzajemnie się wkluczają, co oznacza, że dana encja może reprezentować jeden, konkretny składnik rzeczywistości,
- wszystkie obiekty muszą być wyraźnie identyfikowane, co oznacza, że dana instancja musi się jednoznacznie wyróżniać od pozostałych instancji tego typu.

Z tego też powodu, warto rozróżnić pojęcie **encji / typu encji** od **wystąpienia / instancji encji**. Konkretnie obiekty, jakie pojawiają się w świecie, są określane mianem instancji. Przykładowo encją będzie *Politechnika*, zaś jej instancją będzie jedna konkretna uczelnia techniczna, na przykład *Politechnika Rzeszowska*.

Atrybut

Do stworzenia encji, poza nazwą, wymagany jest jej przynajmniej jeden atrybut (identyfikator). Nie ma górnego ograniczenia, co do ilości atrybutów encji.

Atrybut (ang. *attribute*) to pewnego rodzaju cechy charakterystyczne. Atrybuty dzielą się na dwa typy:

- identyfikatory (klucze) oraz
- deskryptory.

Identyfikatory (klucze) charakteryzują się tym, że pozwalają zidentyfikować wystąpienie encji. Przykładem jest istnienie dwóch różnych samochodów o tych samych właściwościach, np. producent, model itd. W rozróżnieniu ich konieczne jest ustalenie głównego identyfikatora.

Identyfikatorem może być zarówno pojedynczy atrybut, jak i grupa atrybutów. Wśród identyfikatorów wyróżnić można zarówno te naturalne, np. numer dowodu, PESEL, numer rejestracyjny, jak i specjalnie stworzone dla systemu, np. numer pozycji danego produktu w katalogu lub wygenerowany kod. Na przykład, na studiach może zdarzyć się, że kilku studentów posiada takie samo imię i nazwisko – które są atrybutami, jednak ich identyfikatory, w postaci na przykład numeru albumu – wyróżniają każdego z nich z całej reszty studentów.

Warunkiem przy tworzeniu encji jest niewystępowanie takich samych cech o tej samej nazwie. Dodatkowo istotne jest przyporządkowanie do atrybutu **ograniczenia** oraz określonego **typu danych**. **Ograniczenie** informuje m.in. o tym, czy atrybut:

- - jest identyfikatorem(#),
- - zawiera wartość wymaganą (*),
- - zawiera wartość opcjonalną (o).

Przykład encji z określonymi atrybutami pokazany jest na Rys. 19.

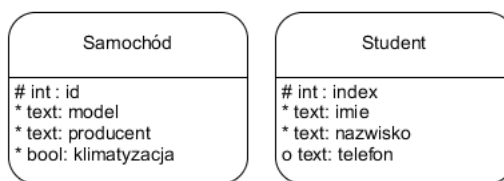


Rys. 19. Przykład encji z określonymi atrybutami

Typ danych pozwala zwiększyć ilość wiedzy o strukturze encji w dalszych etapach tworzenia projektu (implementacja). To, czy dany atrybut przechowuje wartość określonego formatu, to cenna informacja. Przy wyborze cechy atrybutu należy przeanalizować skutki tej decyzji. Przykładowe typy danych z Oracle Data Modeler to:

- BigInt,
- Double,
- Date,
- DateTime,
- Integer.

Przykładowe encje, w których określone są typy trybutów, pokazane są na Rys. 20.



Rys. 20. Przykład encji z określonymi atrybutami i typami

Mianem **atrybutów deskrypcyjnych**, czy też **deskryptorów**, określane są takie atrybuty, które nie są identyfikatorami. Wartości te mogą być, w zależności od potrzeb informacyjnych, obowiązkowe lub też opcjonalne.

Istotnym elementem w planowaniu encji jest kontekst w stosunku do tworzonego systemu. Nie jest możliwe zapisanie wszystkich cech rzeczywistego obiektu. Dlatego należy się skupić na tych, które są potrzebne do funkcjonowania tworzonego systemu informatycznego.

Przykład tworzenia encji:

- Samochód (ogólnie) - pojazd silnikowy, służący do przewozu osób lub ładunków. Istotne informacje o nim to model, producent, rok produkcji, przebieg itd.
- Samochód (inny kontekst systemu tzn. system obsługi mechanik) – podobnie jak w ogólnym kontekście, ale mogą być potrzebne atrybuty dotyczące szczegółów, np. rodzaj silnika.

Związek (relacja)

Związek, czy też relacja (ang. *relationship*) jest pewnego rodzaju logicznym powiązaniem, jakie występuje między poszczególnymi, istniejącymi encjami. Jest to informacja, którą należy przechowywać w systemie, ponieważ nie jest możliwe jej wywnioskowanie, pozyskanie, czy obliczenie z wartości atrybutów danego bytu. Związek pozwala powiązać ze sobą dwie lub więcej występujące w systemie encji. Każda z relacji musi zostać właściwie nazwana tak, aby modele ER były czytelne i zrozumiałe dla ludzi. Związki (relacje) przedstawiają stan ze świata rzeczywistego, np.

student jest właścicielem Indeksu. W procesie implementacji pozwalają one tworzyć powiązania pomiędzy danymi zapisywanymi w systemie.

Cechą charakterystyczną każdego związku jest jego:

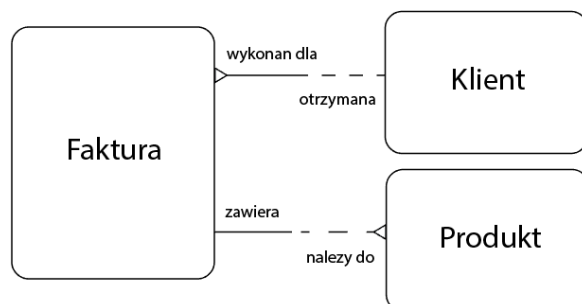
- **liczebność** – inaczej nazywana **typem związku**, oraz
- **opcjonalność**.

Wyróżnia się trzy typy związków (relacji):

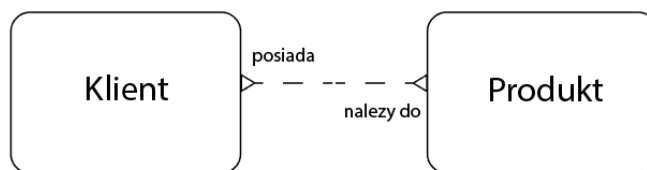
- - jeden do wielu 1 : N,
- - wiele do wielu N : N,
- - jeden do jednego 1 : 1.

Związek w diagramach oznaczany jest najczęściej strzałką, bądź też linią przerywaną lub ciągłą, co jest zależne od cechy jaką jest opcjonalność. Związek powinien być opisany za pomocą jego nazwy, umieszczonej na linii. Opis ma formę czasownika w czasie teraźniejszym, wskazującego, czego dotyczy związek pomiędzy obiektami. Zakończenie linii wskazuje z kolei na liczebność danego związku.

Przykładowe diagramy z relacjami jeden do wielu oraz wiele do wielu pokazane są na Rys. 21 i Rys. 22, odpowiednio. Przedstawiają one następujące sytuacje. Faktura należy do klienta, który może mieć ich dowolną liczbę (żadnej 0, co oznaczone jest, linią przerywaną, lub wiele). Faktura musi dotyczyć jednego klienta (jednego i tylko jednego). Faktura obejmuje (zawiera) wiele produktów i musi mieć przynajmniej jeden z nich. Produkt może dotyczyć (należać do) wielu faktur lub żadnej. W relacji wiele do wielu Klient może posiadać 0 lub wiele produktów, podobnie produkt może należeć do nikogo lub wielu Klientów.



Rys. 21. Diagram ERD z relacjami jeden do wielu



Rys. 22. Diagram ERD z relacją wiele do wielu

W literaturze, wyróżnić można kilka stopni związków, które zależne są od liczby encji, jakie biorą w nich udział. Są to między innymi relacje: **dwuargumentowe**, **jednoargumentowe**, **trzyargumentowe**, czy także **wieloargumentowe**. Najczęściej stosowanymi i rozwiązanymi są pierwsze związki, nazywane również binarnymi. Budowane są na bazie dwóch różnych obiektów, między którymi pojawia się pewna zależność. Dla odróżnienia, związki jednoargumentowe, nazywane również unarnymi lub rekurencyjnymi, mają wyłącznie jednego uczestnika, czyli także jedną relację. Powstają one w podobnie do związków binarnych, z taką różnicą, że encja jest tylko jedna – obiekt

jest powiązany sam ze sobą. W takich powiązaniach występować mogą relacje jeden do wielu oraz wiele do wielu. Związki trzyargumentowe, czyli ternarne oraz ostatnie wymienione, nazywane również n-arnymi, prezentują powiązania jakie występują między więcej niż dwoma połączonymi ze sobą encjami. Nie mogą one być bezpośrednio wprowadzane do relacyjnych baz, ponieważ wymagają specjalnych przekształceń.

4.3 Budowa modelu danych

Budowa modelu danych obejmuje kilka etapów:

1. Identyfikacja zbioru encji wraz z ich atrybutami kluczowymi
2. Identyfikację bezpośrednich zależności między encjami
3. Weryfikacja diagramu ERD pod kątem zawartości encji i poprawności związków

Ad 1. Identyfikacja zbioru encji wraz z ich atrybutami kluczowymi

Analizę i tworzenie diagramu należy rozpocząć do wytypowania obiektów (encji), które system ma obsługiwać. Może to być samochód, faktura, osoba lub mniej oczywiste obiekty jak rola, stan, czynność. Następnie należy uzupełnić encje systemu o ich atrybuty. Szczegóły obiektów są niezwykle ważne, ponieważ poszerzają wiedzę odnośnie ewentualnego działania systemu.

Ad 2. Identyfikację bezpośrednich zależności między encjami

Kolejnym etapem po zidentyfikowaniu encji jest powiązanie ich ze sobą relacjami, które prezentują, jak ma funkcjonować system od strony budowy modelu. Polega to na wyszukaniu bezpośrednich powiązań pomiędzy wszystkimi parami (A, B) obiektów w systemie. W tym celu należy sprawdzić, czy każde wystąpienie obiektu typu A może być powiązane z pewnym wystąpieniem obiektu typu B, bez udziału innego obiektu. Należy przy tym wziąć pod uwagę jedynie powiązania istotne z punktu widzenia systemu, to znaczy takie, o których informacja powinna być utrzymywana w systemie, ponieważ odzwierciedlają one reguły i zasady panujące w organizacji/instytucji użytkownika.

Po ustaleniu powiązań należy zbadać ich cechy charakterystyczne, w szczególności liczebność i opcjonalność. Pozwala to na sporządzenie wstępnego diagramu encji.

Przykład

Załóżmy, że mamy opracować model danych dla prostego systemu przetwarzania zamówień w pewnej firmie, obsługującej klientów, zamawiających towar produkowany przez tą firmę. Wyroby oferowane przez firmę są gromadzone w magazynach, zlokalizowanych w różnych miejscach.

Encje i atrybuty kluczowe dla tego systemu pokazane są w Tab. 13.

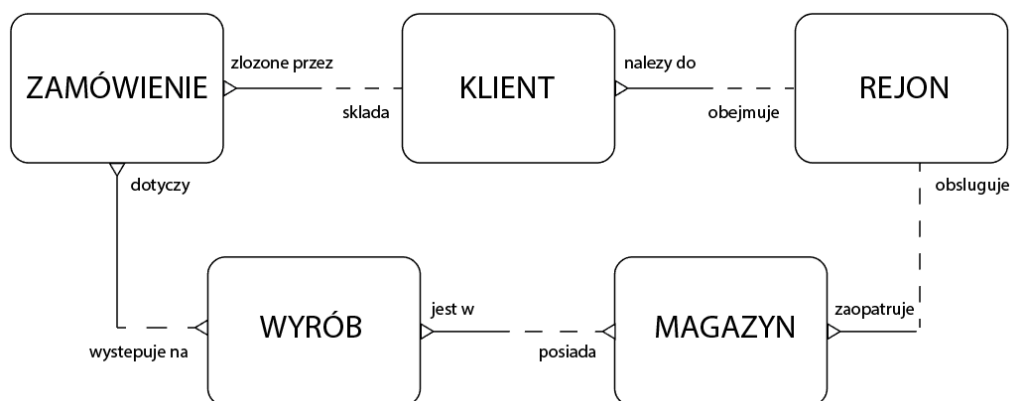
Tab. 13. Encje i atrybuty kluczowe dla przykładowego systemu zamówień

Encja	Atrybut
Klient	Numer klienta
Wyrób	Kod wyrobu
Magazyn	Kod magazynu
Rejon	Kod rejonu
Zamówienie	Numer zamówienia

Powiązania pomiędzy encjami dla tego systemu przedstawia Tab. 14, w której znakiem X oznaczono istnienie bezpośredniego, istotnego powiązania. W wyniku analizy ich cech charakterystycznych – liczebności i opcjonalności, sporządzić można wstępny diagram związków encji (Rys. 23).

Tab. 14. Tablica krzyżowa encji dla przykładowego systemu zamówień

	Klient	Wyrób	Rejon	Magazyn	Zamówienie
Klient			X		X
Wyrób				X	X
Magazyn				X	
Rejon					
Zamówienie					



Rys. 23. Wstępny diagram związków encji

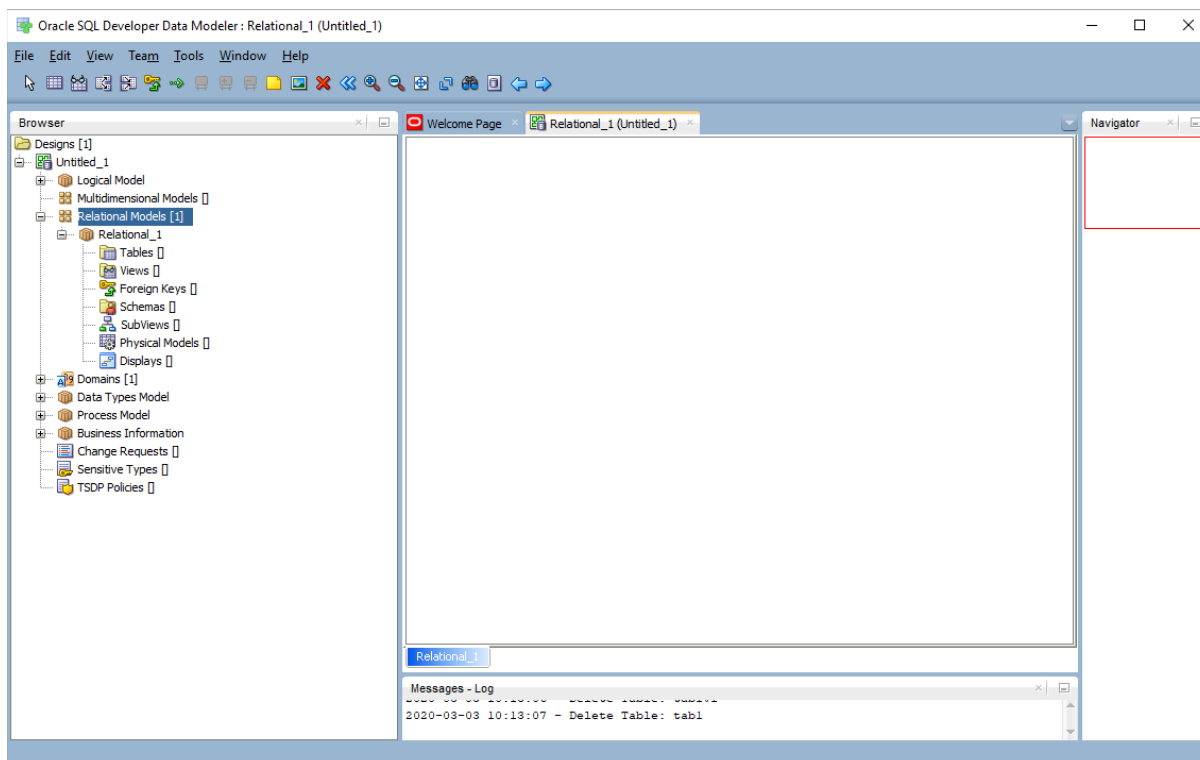
Encje wraz z określonymi ich atrybutami dla naszego przykładu zebrane zostały w Tab. 15.

Tab. 15. Atrybuty encji

Encja	Atrybut	Opis	Typ	Atrybut kluczowy	Atrybut opcjonalny
MAGAZYN	Nr_mag	Numer magazynu	INTEGER	X	
	Nazwa_mag	Nazwa magazynu	VARCHAR2(64)		
	Adres_mag	Adres magazynu	VARCHAR2(64)		
REJON	Kod_rej	Kod rejonu	INTEGER	X	
	Nazwa_rej	Nazwa rejonu	VARCHAR2(64)		
KLIENT	Id_kl	Identyfikator klienta	INTEGER	X	
	Nazwa_kl	Nazwa klienta	VARCHAR2(64)		
	Adres_kl	Adres klienta	VARCHAR2(64)		
ZAMÓWIENIE	Id_zam	Identyfikator zamówienia	INTEGER	X	
	Data_wyst	Data wystawienia	DATE		
	Zam_og	Wartość zamówienia	NUMBER(6,2)		
WYRÓB	Kod_wyr	Kod wyrobu	INTEGER	X	
	Nazwa_wyr	Nazwa wyrobu	VARCHAR2(64)		
	Cena_jedn	Cena jednostkowa	NUMBER(6,2)		

4.4 Wprowadzenie do programu Data Modeler

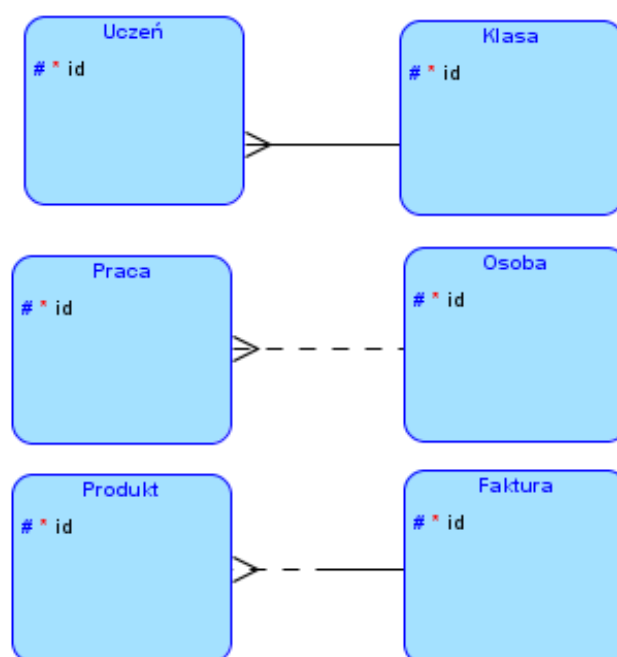
Narzędzie Data Modeler [5] umożliwia projektowanie diagramu ERD. Nowy diagram jest tworzony na podstawie struktury „Relational Models”. Możliwe jest tworzenie nowych encji w postaci tabel, relacji między nimi. Okno główne programu Data Modeler pokazane jest na Rys. 24.



Rys. 24. Okno główne programu Data Modeler

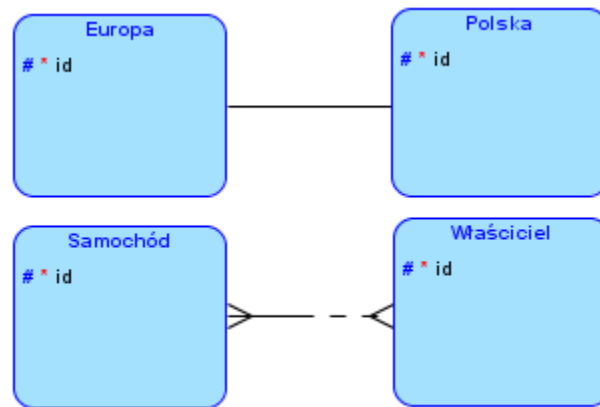
Tworzenie diagramu ERD możliwe jest w panelu Logical Model. Tam możliwe jest tworzenie encji, modyfikowanie jej właściwości oraz wiązanie obiektów relacjami. Przycisk dodawania nowej encji jest dostępny w górnej belce. Po umieszczeniu encji w polu projektowania pojawi się formularz pokazany na Rys. 25. Należy określić nazwę encji oraz jej atrybuty. Atrybuty są dostępne w zakładce Attributes. Istotne jest wybranie typu danych Logical oraz Identyfikatora obsługującego daną encję. Po stworzeniu potrzebnych encji i ich atrybutów należy powiązać poszczególne obiekty relacjami. W górnej belce dostępne jest tworzenie relacji (zielone strzałki). Po powiązaniu dwóch encji możliwe jest modyfikowanie relacji pod kątem kierunku relacji oraz opcjonalności. Na Rys. 26 umieszczone są przykłady relacji w notacji Barkera.

Rys. 25. Formularz tworzenia encji w narzędziu Data Modeler



Rys. 26. Przykład relacji na diagramie ERD, utworzonym za pomocą programu Data Modeler

Szczególnymi relacjami są relacje 1:1 oraz N:N (wiele do wielu). Relacje jeden do jednego stosuje się w specyficznych sytuacjach, często związanych z aspektami technicznymi systemu. Związek wiele do wielu występuje częściej i można go stosować w diagramach ERD typu ogólnego. Wymaga on jednak określonego postępowania na etapie implementacji i projektowania (normalizacji). Przykłady takich relacji pokazano na Rys. 27.



Rys. 27. Przykłady relacji 1:1 i N:N (Data Modeler)

5 Diagramy sekwencji

5.1 Wstęp

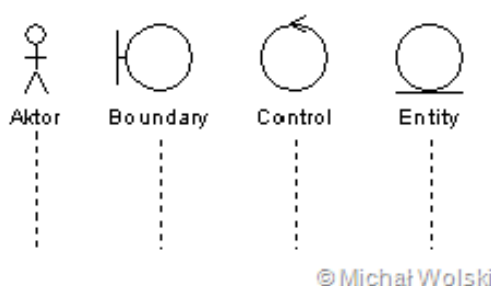
Istotną rolę w zaprezentowaniu działania systemu jest opisanie interakcji, które występują pomiędzy występującymi w nim obiektami. **Diagram sekwencji** (ang. sequence diagram) umożliwia modelowanie komunikacji odbywającej się w czasie. Opis zachodzących w tym procesie interakcji reprezentuje sekwencje konkretnych scenariuszy. Jest to więc naturalne rozszerzenie diagramu przypadków użycia. Szczegółowo odpowiada na pytanie, jak wykonuje się dany przypadek i jaka komunikacją wchodzi w jego skład. Diagram zawiera obiekty umieszczone wzdłuż osi X oraz komunikaty przesyłane wzdłuż osi Y. Głównymi elementami diagramu sekwencji są:

- Linia życia,
- Komunikat,
- Fragment.

Każdy z tych elementów został opisany w kolejnych podrozdziałach.

5.2 Linia życia

Linia życia opisuje czas istnienia bytu będącego uczestnikiem sekwencji podczas realizacji scenariusza. Czas życia bytu przedstawiany jest jako długość linii przerywanej na osi Y. Bytem może być obiekt, system, aktor itd. Rodzaj bytu określa jednocześnie rolę, jaką pełni on w systemie (zob. Rys. 28). Byt będący aktorem informuje, że pełni on funkcję zewnętrzną w stosunku do systemu. Obiekt klasy granicznej reprezentuje interfejs między systemem a bytami występującymi poza nim. Obiekt klasy sterującej używany jest do sterowania działaniem jednej lub wielu klas. Natomiast obiekt klasy danych stosowany jest do reprezentowania danych, które muszą być zachowane w systemie.

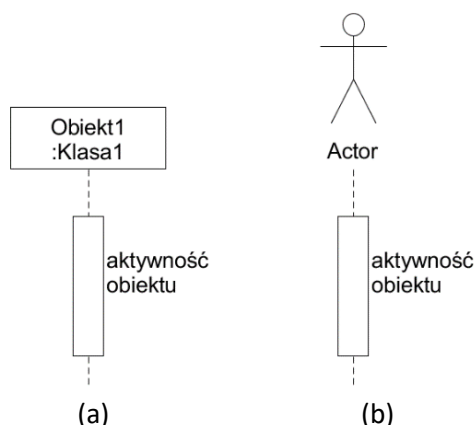


Rys. 28. Byty linii życia różnych typów [źródło: https://wolski.pro/wp-content/uploads/2012/03/najczesciej_stosowana_notacja_UML_2011_html_m4c0aae15.gif]

Linii życia z symbolem elementu aktora jest używana, gdy określony diagram sekwencji jest własnością przypadku użycia. Linia życia z elementem encji reprezentuje dane systemowe. Linia życia z elementem granicznym oznacza granicę systemu. Granicę taką stanowią na przykład ekrany interfejsu użytkownika, bramki baz danych lub menu, z którymi użytkownicy wchodzi w interakcję. Linia życia z elementem kontrolnym oznacza jednostkę kontrolującą lub kierownika, który organizuje i planuje interakcje pomiędzy granicami i bytami oraz pełni rolę mediatora pomiędzy nimi.

Aktywność bytu na linii życia wskazywana jest za pomocą prostokąta, zwanego paskiem aktywacji.

Przykładowe linie życia pokazano na Rys. 29.



Rys. 29. Przykłady linii życia obiektów *Obiekt1* klasy *Klasa1* (a) i *Aktor* (b)

5.3 Komunikat

Komunikat (ang. message) prezentuje informację przesyłaną pomiędzy obiektami na linii życia. Komunikat najczęściej reprezentuje komunikację synchroniczną, czyli cechującą się oczekiwaniem na odpowiedź i zwróceniem wyniku. Komunikacja może być też asynchroniczna, czyli taka, w której nie ma wymogu oczekiwania na odpowiedź. W przypadku wystąpienia komunikacji wewnętrznej interakcja ma kierunek skierowany do tego samego obiektu.

Użycie na diagramie komunikatu synchronicznego wiąże się z założeniem, że natychmiast po jego wywołaniu przychodzi odpowiedź. Taka konwencja pozwala na zmniejszenie liczby elementów na diagramie. W sytuacji, gdy modelowany fragment rzeczywistości jest inny niż założenia konwencji, należy umieścić komunikat zwrotny.

W diagramie sekwencji komunikaty oznaczane są strzałkami. Kierunek strzałki określa nadawcę i odbiorcę komunikatu. Opis na strzałce (podpis) pozwala opisać komunikat. Różne groty wskazują na różne rodzaje wysyłanej/otrzymywanej wiadomości.

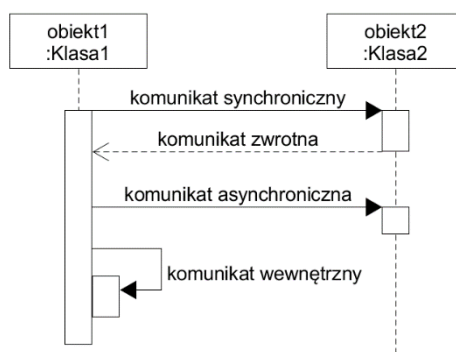
Format podpisu jest następujący:

atrybut = message_name (argumenty): return_type

Wszystkie części oprócz *message_name* są opcjonalne.

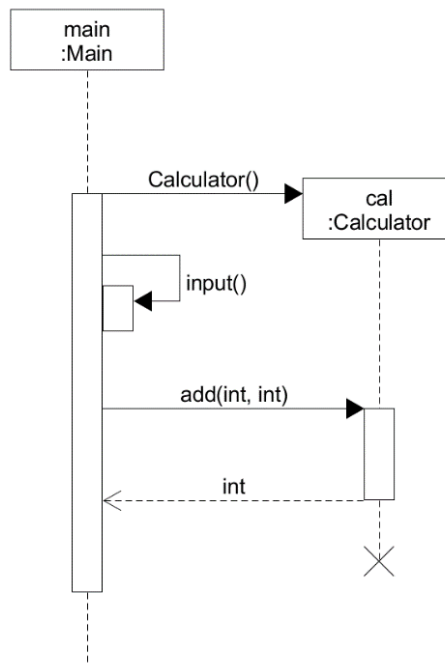
Komunikat zwrotny oznaczany jest strzałką narysowaną linią przerywaną.

Przykładowe komunikaty pokazano na Rys. 30.



Rys. 30. Przykłady komunikatów między obiektami *obiekt1* klasy *Klasa1* i *obiekt2* klasy *Klasa2*

Byty mogą być powoływane do życia i usuwane w czasie trwania sekwencji zdarzeń przez już istniejące byty, w czasie ich aktywności. Utworzenie nowego bytu i jego usunięcie możliwe jest z pomocą komunikatów. Warunkiem poprawnego zniszczenia obiektu jest zakończenie aktywnej komunikacji na linii życia. Usunięty byt oznaczany jest w diagramie za pomocą krzyżyka kończącego linię życia z nim związaną. Przykładowy diagram zawierający sekwencję, w trakcie której byt *main* powołuje do życia byt *cal*, a następnie go usuwa pokazany jest na Rys. 31.



Rys. 31. Diagram sekwencji z tworzonym przez byt *main* obiektem *cal* i jego usunięciem po wykonaniu swojego zadania

5.4 Fragment

Fragment (ang. combined fragment) to wydzielona, zamknięta część diagramu sekwencji, umożliwiająca rozszerzenie możliwości obejmowanego przez siebie obszaru. Przykładami operacji wykonywanymi we fragmencie są np. pętla, powtórzenie, alternatywne scenariusze. Fragment może też wskazywać poziom abstrakcji modelowanej części systemu.

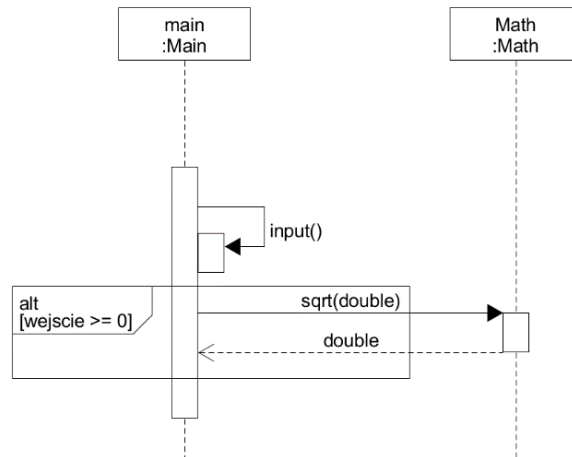
Rodzaj fragmentu jest określany poprzez umieszczenie odpowiedniego słowa kluczowego w lewym górnym rogu. Najbardziej popularne są następujące typy:

- alt – alternatywny scenariusz (dzieli fragment interakcji zgodnie z warunkami logiki Boole’a na dwa alternatywne scenariusze; każda z alternatyw musi być opatrzona warunkiem doboru, którego spełnienie gwarantuje wykonanie danej alternatywy),
- loop – powtórzenie interakcji w pętli określoną warunkiem liczbę razy,
- par – równoległe wykonywanie przepływu wiadomości.

Scenariusz alternatywny jest stosowany w przypadku konieczności dokonania wyboru pomiędzy dwoma lub więcej sekwencjami komunikatów. Modeluje on logikę “if – then – else”. Alternatywny fragment jest reprezentowany przez duży prostokąt lub ramkę, z warunkiem umieszczonym w lewym górnym rogu, po słowie *alt*.

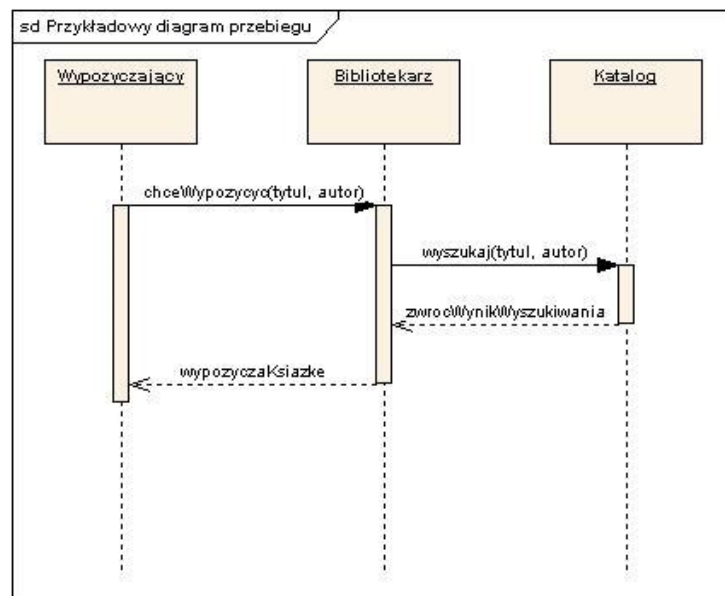
Aby pokazać dwie lub więcej alternatyw, większy prostokąt jest następnie dzielony na tzw. operandy interakcji za pomocą linii przerywanej. Każdy operand jest powiązany z odpowiadającą mu ścieżką wyboru, określoną w lewym górnym rogu związanej z nim części fragmentu.

Diagram sekwencji zawierający fragment ze scenariuszem alternatywnym pokazano na Rys. 32.



Rys. 32. Diagram sekwencji z fragmentem korzystającym ze scenariusza alternatywnego

Przykładowy diagram sekwencji prezentujący próbę wypożyczenia książki pokazano na Rys. 33. Jako komunikat wejściowy często stosuje się podanie funkcji/metody wraz z informacją o parametrach wejściowych (nazwie lub typie). Komunikatem wyjściowym może być odpowiedź, np. zwracana wartość reprezentowana przez nazwę zmiennej lub typ.



Rys. 33. Diagram sekwencji dla próby wypożyczenia książki

6 Diagramy klas

6.1 Wstęp

Diagramy klas (ang. class diagram) to najczęściej występujące i zarazem kluczowe diagramy w modelach obiektowych. Są one wykorzystywane do modelowania statycznych relacji pomiędzy komponentami systemu i stanowią podstawę przyszłej, obiektowej bazy danych. Pojedynczy model UML może posiadać wiele diagramów klas, pokazujących ten sam system z różnych perspektyw (np. koncepcyjnej, implementacyjnej).

Diagram klas pozwala przestawić statyczne elementy systemu, takie jak zbiory klas, czy interfejsy, oraz związki między nimi. Diagram taki może prezentować wszystkie elementy systemu lub tylko ich część. Stanowi on wsparcie w tworzeniu funkcji oraz głównych elementów całego systemu. Oznacza to, że informacje w nim zawarte mogą być poglądowe. Należy je sformułować tak, aby były jak najbardziej pomocne. Ponieważ diagram nie jest reprezentacją całego kodu, nie muszą być w nim zaprezentowane wszystkie elementy, ani wszystkie ich właściwości. Wobec tego brak jakiegoś elementu na diagramie nie oznacza, że odpowiadająca mu część systemu nie powinna istnieć. Elementy, które nie pełnią istotnej roli w opisie funkcjonowania systemu, nie muszą być przedstawione na diagramie.

W diagramach klas stosuje się przedstawione dalej notacje oraz elementy.

6.1.1 Klasa

Klasę można zdefiniować jako uogólniony zbiór obiektów, które mają takie same atrybuty, operacje, związki i znaczenie. Pojedyncza klasa reprezentuje co najmniej jeden obiekt w działającym systemie. Klasa jest podstawowym elementem diagramu klas. Jest na nim przedstawiana jako prostokąt złożony z trzech podstawowych sekcji:

- – nazwy klasy,
- – zestawu atrybutów,
- – zestawu operacji.

W zależności od szczegółowości diagramu nie wszystkie sekcje klasy muszą być na nim pokazane, w szczególności może wystąpić tylko nazwa klasy tzw. klasa anonimowa. Jest to zwłaszcza przydatne na poziomie diagramów koncepcyjnych.

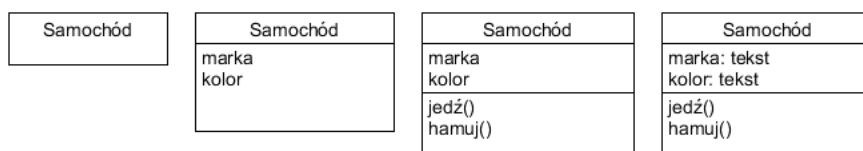
Nazwa klasy jest określana przy pomocy rzeczownika. Zwykle składa się z liter oraz może zawierać cyfry. Choć dopuszczalne jest stosowanie innych znaków, to przyjęte jest, że można stosować dodatkowo tylko znak podkreślenia '_'.

Zapis **atrybutów** odbywa się wg następującej formuły (w [] umieszczono elementy opcjonalne):

```
[widoczność] nazwaAtrybutu [liczność] [: typAtrybutu]
```

```
[=wartośćDomyślna] [{właściwość}]
```

Przykład klasy dla obiektu reprezentującego samochód, przedstawionego w postaci, jaką można zawrzeć w diagramie klas, pokazany jest na Rys. 34. Kolejne reprezentacje zawierają opis tej klasy na różnych poziomach szczegółowości.

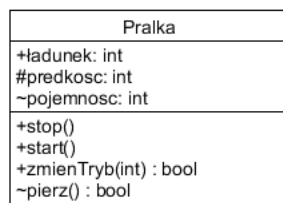


Rys. 34. Przykład opisu klasy reprezentującej samochód, na różnych poziomach szczegółowości

W projektach informatycznych istotne jest dodanie opisu oraz dodatkowych właściwości, takich jak poziom dostępu oraz informacje o typach. Poziom dostępu oznacza się następującymi modyfikatorami:

- + – oznacza składnik publiczny (public)
- # – oznacz składnik chroniony (protected)
- – – oznacza składnik prywatny (private)
- ~ – oznacz składnik dostępny w obrębie projektu (package)
- metody abstrakcyjne w klasie abstrakcyjnej są *pochylone* lub podkreślone.

Typy zawarte w diagramie mogą bazować na określonych językach programowania, np. Java: int, string, float, lub być abstrakcyjne np. liczba, tekst itd. Przykład szczegółowego opisu klasy „Pralka” pokazany jest na Rys. 35.



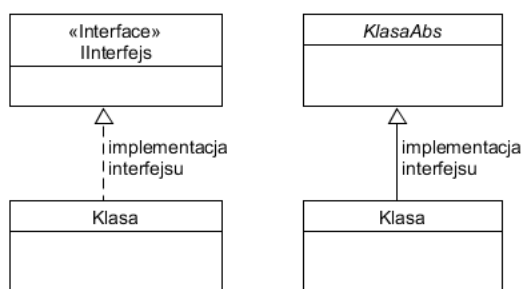
Rys. 35. Przykład szczegółowego opisu klasy „Pralka”

6.1.2 Klasa abstrakcyjna

Klasa abstrakcyjna przedstawiana jest podobnie jak klasa, z tym, że jej nazwa pisana jest kursywą. Klasa dziedzicząca po klasie abstrakcyjnej musi implementować metody abstrakcyjne. W celu rozróżnienia metod dziedziczonych i implementowanych ich nazwy pisze się kursywą.

6.1.3 Interfejs

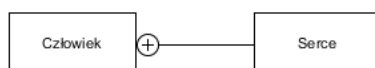
Interfejs przedstawiany jest podobnie jak klasa, z tym, że jego nazwa poprzedzona jest słowem kluczowym <<interface>>. Dodatkowym ułatwieniem jest dodawanie do nazwy interfejsu litery I. Klasa implementowana jako interfejs musi implementować zawarte w niej metody. Przykład reprezentacji interfejsu w diagramie klas pokazany jest na Rys. 36. Strzałki z pustymi grotami skierowane są do Interfejsu i Klasy Abstrakcyjnej. Rodzaj relacji rozróżnia się przez ciągłą lub przerywaną linię.



Rys. 36. Przykład reprezentacji interfejsu

6.1.4 Klasa zagnieżdżona

Klasa zagnieżdżona to klasa umieszczona w innej klasie. Przykład klasy zagnieżdżonej pokazany jest na Rys. 37.



Rys. 37. Klasa zagnieżdżona

6.2 Relacje

Relacja określa rodzaj związku między klasami. Istotną rolę w opisywaniu relacji jest określenie cechy krotności. Obrazuje się ją na końcach linii reprezentującej relację. Przykłady krotności to:

- 1 – oznacza dokładnie jeden obiekt
- 0..3 – oznacza od zera do trzech obiektów
- * – oznacza dowolną ilość obiektów
- 3..* – oznacza od trzech do dowolnej ilości obiektów

Przy braku informacji o krotności przyjmuje się domyślnie, że wynosi ona jeden. Przykład klas powiązanych relacją jeden do wielu przedstawiony jest na Rys. 38.



Rys. 38. Przykład relacji

Wyróżnia się następujące relacje:

- **Zależność** – jest to najsłabsza z relacji. Istnieje, gdy jedna klasa wykorzystuje chwilowo inną klasę lub jest świadoma jej istnienia. Przykładem zależności jest użycie obiektu jako parametru w metodzie innej klasy. Zmiana w obrębie jednej klasy może, ale nie musi dokonywać zmian w drugiej klasie.
- **Asocjacja** – oznacza relację silniejszą niż zależność. Klasy powiązane asocjacją nie wpływają na siebie i na swój cykl życia, nie zarządzają sobą, nie tworzą, są powiązane do momentu usunięcia. Asocjacja wskazuje jedynie na to, że jeden obiekt jest związany z innym. Często jest mylona z agregacją.

- **Agregacja częściowa** – jest to mocniejsza odmiana asocjacji. Pokazuje, że jedna część może należeć do drugiej, przy czym usunięcie jednej z części nie wpływa na drugą. Przykładem tej relacji jest przechowywanie referencji w obiekcie. Usunięcie części bazowej nie usuwa części, na którą wskazują referencje.
- **Agregacja całkowita (kompozycja)** – obrazuje sytuację zbliżoną do agregacji częściowej z tym, że klasa bazowa kontroluje cykl życia części, z którą jest powiązana. W tego typu relacji usunięcie głównego obiektu usuwa również obiekt klasy, z którą jest połączony relacją agregacji całkowitej.
- **Uogólnienie (dziedziczenie)** – pozwala stworzyć hierarchie klas, od ogólnych do bardziej szczegółowych. Przykładami uogólnienia są dziedziczenie oraz wykorzystanie interfejsów. Uogólnienie pozwala skrócić kod i organizować go od strony logicznej.

Charakterystyczne cechy poszczególnych rodzajów relacji oraz odpowiadające im oznaczenia w diagramach klas pokazane są w postaci schematu na Rys. 39.



Rys. 39. Schemat relacji, ich znaczeń oraz związku

Przykłady różnych relacji między klasami pokazane są na Rys. 40.

6.3 Proces tworzenia diagramu klas

Proces tworzenia diagramu klas składa się z następujących etapów, wykonywanych iteracyjnie:

1. Zidentyfikowanie i nazwanie klas
2. Opcjonalne określenie zobowiązań klas
3. Połączenie poszczególnych klas związkami asocjacji
4. Wypełnienie asocjacji przy pomocy cech (nazw, ról, nawigacji, itd.)
5. Opracowanie innych rodzajów związków (uogólnień, zależności, realizacji)

Przykładowy diagram klas przedstawiony jest na Rys. 41.

6.4 Mapowanie obiektowo-relacyjne (ORM)

W praktyce powszechne jest wykorzystanie relacyjnych baz danych. Systemy zarządzania tego rodzaju bazami są szybkie, stabilne i bezpieczne. Problemem jest przełożenie logicznej obiektowej struktury systemu na relacyjną bazę danych. Nie jest też oczywiste korzystanie z takiej bazy danych w obiektowej aplikacji [6]. Odzworowanie obiektowej architektury systemu informatycznego na bazę danych (lub inny element systemu) o charakterze relacyjnym możliwe jest z użyciem podejścia opartego na **mapowaniu obiektowo-relacyjnym (ORM)** – skrót od ang. Object Relational Mapping). Z podejściem tym związanych jest jednak szereg problemów wydajnościowych.

Do mapowania obiektowo-relacyjnego można wykorzystać narzędzia oparte na standardzie **JPA (Java Persistence API)**, pozwalające zdefiniować odwzorowanie zawartości relacyjnej bazy danych na

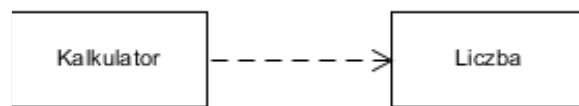
obiekty w używanym przez nas języku programowania oraz wykonywać operacje na danych w bazie danych tak jak na zwykłych obiektach języka programowania.

Standard JPA określa jak ma działać narzędzie do mapowania relacyjno-obiektowego. Różni dostawcy oprogramowania (SUN, IBM, Oracle, twórcy narzędzi OpenSource – Hibernate) stworzyli różne implementacje JPA. Teoretycznie pisana przez nas aplikacja powinna działać tak samo, niezależnie od tego z jakiej implementacji JPA korzystamy. W praktyce nie ma takiej gwarancji – z reguły konieczne są niewielkie poprawki.

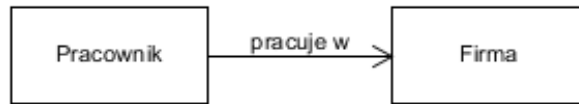
Z punktu widzenia programisty JPA daje możliwość operowania na obiektach zwanych **encjami** oraz zapisywania wyników operacji do relacyjnej bazy danych (za pomocą obiektu *EntityManager*). Sposób w jaki obiekty i ich połączenia przekładają się na elementy bazy danych są definiowane za pomocą adnotacji lub dokumentów XML. Poza standardowym zestawem operacji udostępnianych przez obiekt *EntityManager* standard JPA definiuje język zapytań, **JPA Query Language**, podobny do SQL.

Podstawowe zasady pracy z narzędziami ORM są następujące [6]:

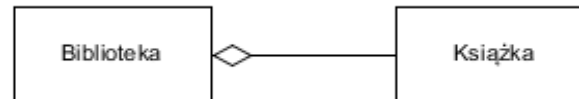
1. Tworzymy model danych w obiektowym języku programowania.
2. Tworzymy schemat bazy danych odpowiadający temu modelowi (może się tu okazać konieczne oparcie się na już istniejącej bazie danych, z którą musimy współpracować).
3. Definiujemy odwzorowanie bazy danych na model relacyjny.
4. Tworzymy aplikację obiektową operującą na modelu obiektowym (tworząc kod aplikacji w zasadzie nie musimy się zastanawiać jak nasze obiekty zostaną zapisane).
5. W razie konieczności pobrania obiektów z bazy danych, bądź utrwalenia jakiegoś nowo utworzonego obiektu, czy też usunięcia utrwalonego obiektu – posługujemy się odpowiednim API danego narzędzia ORM. Jest to jedyne miejsce, w którym w naszej aplikacji zajmujemy się tym, że współpracujemy z jakąś bazą danych.



(a)



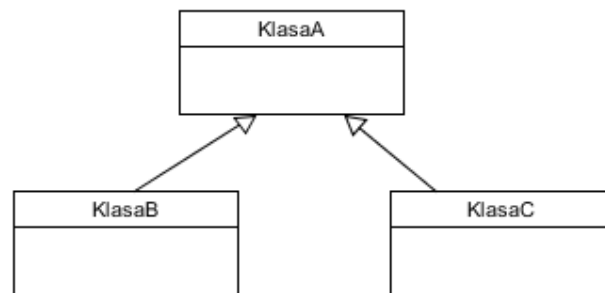
(b)



(c)

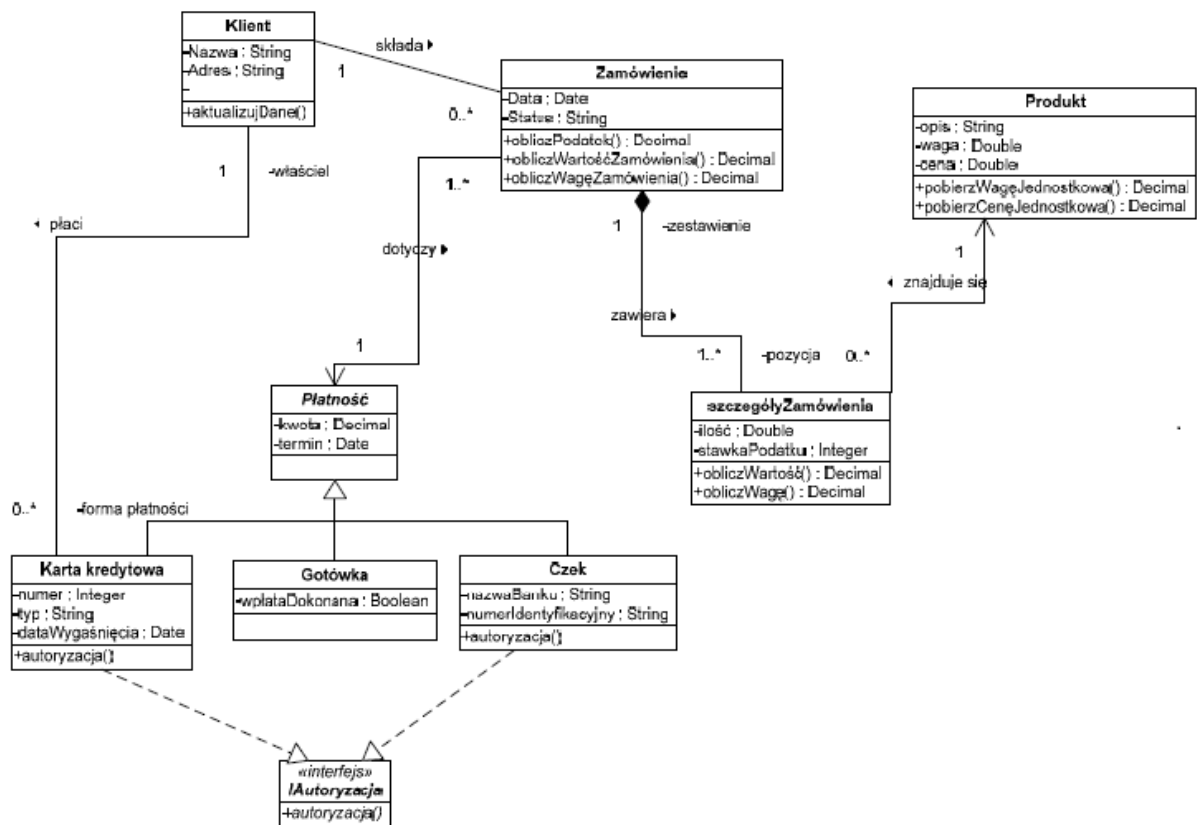


(d)



(e)

Rys. 40. Przykłady relacji między klasami: a) zależność, b) asocjacja, c) agregacja częściowa, d) agregacja całkowita, e) uogólnienie



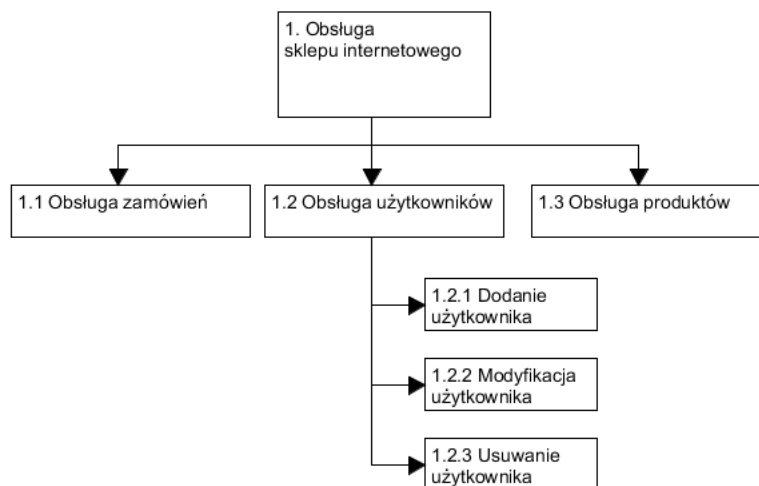
Rys. 41. Przykład diagramu klas dla systemu zamówień

7 Diagram hierarchii funkcji (FHD)

7.1 Wstęp

Diagram hierarchii funkcji (FHD od *ang. Function Hierarchy Diagram*) pozwala na opis funkcji systemu oraz uporządkowanie ich w postaci modelu koncepcyjnego. Jego zadaniem jest przedstawienie, czym zajmuje się system i jakie zadania ma wykonywać. Jest to jeden z początkowych diagramów tworzony w procesie opracowywania wymagań funkcjonalnych systemu. Diagram hierarchii funkcji pozwala prezentować funkcjonalności w postaci uogólnionych, głównych funkcji oraz szczegółowych podfunkcji. Dokładniejszy opis oznacza zbudowanie hierarchii i dekompozycje funkcji np. obsługa klientów => dodanie klienta, usunięcie klienta itd.

Fragment przykładowego diagramu FHD opisującego obsługę sklepu internetowego pokazano na Rys. 42.

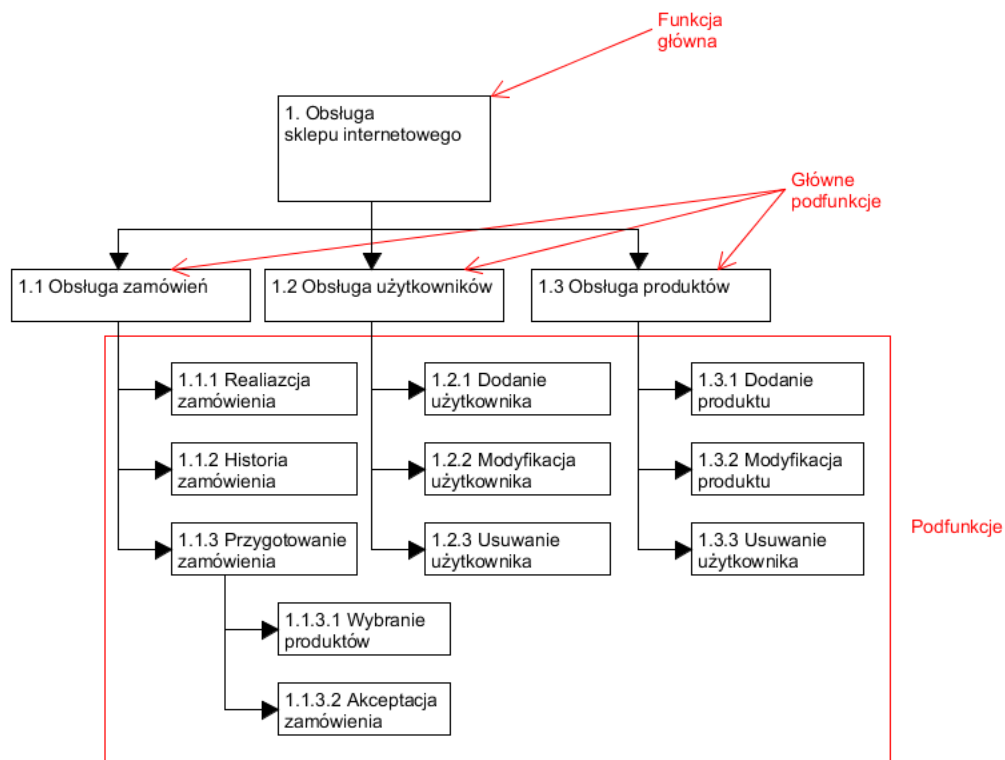


Rys. 42. Fragment diagramu hierarchii funkcji dla systemu obsługującego sklep internetowy

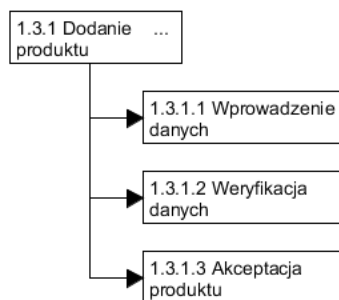
Szczegółowością diagramu FHD można regulować przez dobudowywanie odpowiednich podfunkcji. Przykładem jest funkcjonalność „obsługa użytkownika” z Rys. 42, która została rozbita na trzy podfunkcje.

Możliwe jest dzielenie diagramu na mniejsze części. Istotne jest wówczas, aby numeracja funkcji we wszystkich częściach była zgodna. Przynależność funkcji do ogólnego diagramu oznaczana jest wielokropkiem dodanym do bloku podfunkcji.

Uszczegółowienie funkcji *Dodanie produktu* dla systemu obsługującego sklep internetowy, którego diagram hierarchii funkcji przedstawiono na Rys. 43a), pokazano na Rys. 43b).



(a)

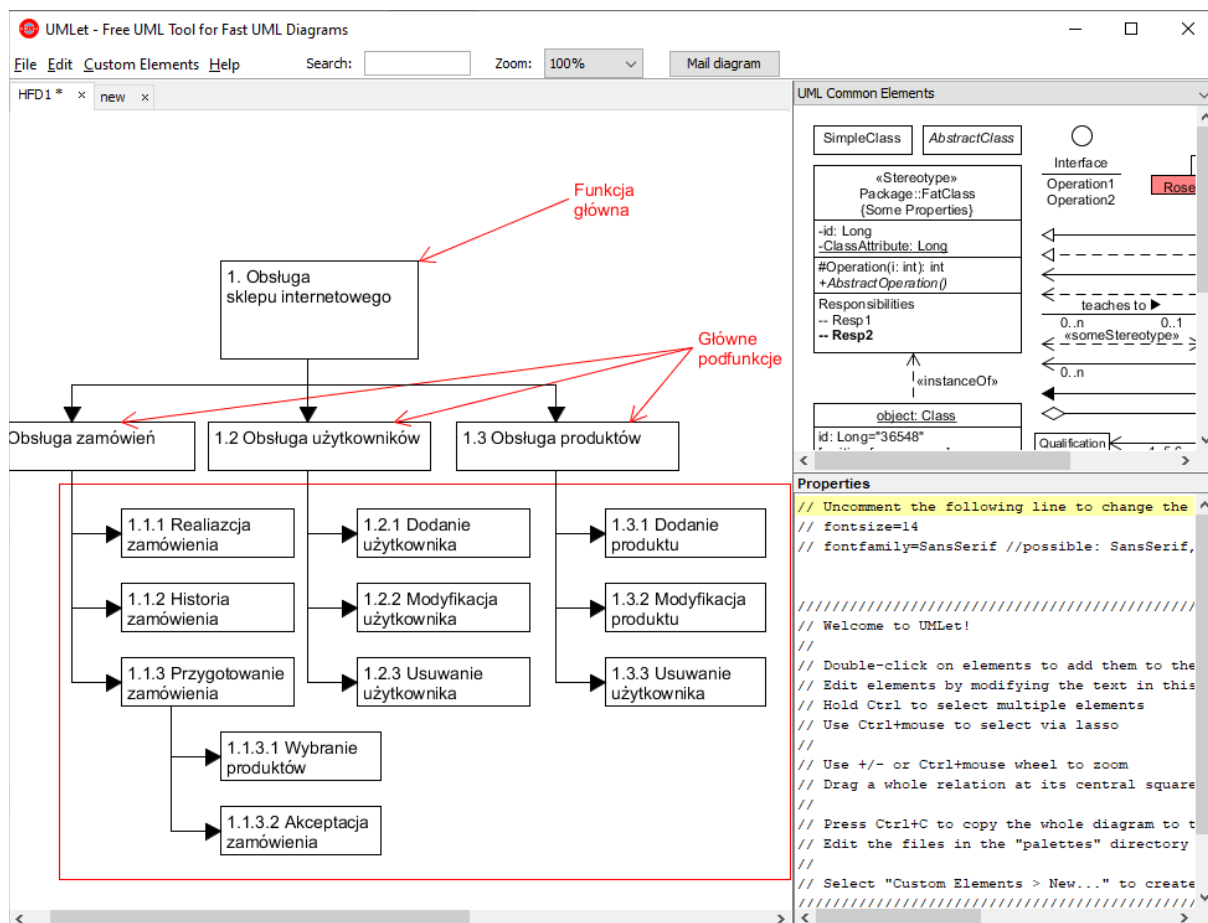


(b)

Rys. 43. Diagram hierarchii funkcji dla systemu obsługującego sklep internetowy (a) z uszczegółowioną funkcją 1.3.1 Dodanie produktu (b)

7.2 Wprowadzenie do tworzenia diagramu FHD

Do wykonania diagramu FHD można wykorzystać narzędzie UMLet [2]. Okno narzędzia z przykładowym diagramem FHD pokazano na Rys. 44.



Rys. 44. Okno programu UMLet z przykładowym diagramem FHD

8 Diagram przepływu danych

8.1 Wstęp

Diagram przepływu danych (DFD od *ang. Data Flow Diagram*) pozwala na graficzne zaprezentowanie przepływu danych pomiędzy elementami systemu. W dalszej części rozdziału opisano następujące zagadnienia:

- Podział diagramów DFD na kategorie
- Elementy diagramu DFD
- Tworzenie diagramów DFD

8.2 Rodzaje diagramów DFD

Diagramy DFD można podzielić z uwzględnieniem szczegółów, jakie opisują na następujące kategorie:

- Diagram kontekstowy – najbardziej ogólny typ reprezentacji, obrazujący granice systemu, źródła i odbiorców przepływów oraz jego główne wyjście i wejście,
- Diagram systemowy (zerowy) – przedstawienie głównych funkcji systemu,
- Diagram szczegółowy (procesy elementarne) – pokazuje szczegółową realizację procesów głównych.

8.3 Elementy diagramu DFD

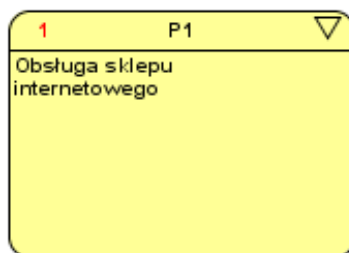
Diagramy DFD buduje się z następujących składowych:

- Funkcja / Proces,
- Przepływ,
- Magazyn danych,
- Obiekt zewnętrzny / Agent.

Poniżej opisano każdy z tych elementów.

8.3.1 Funkcja / Proces

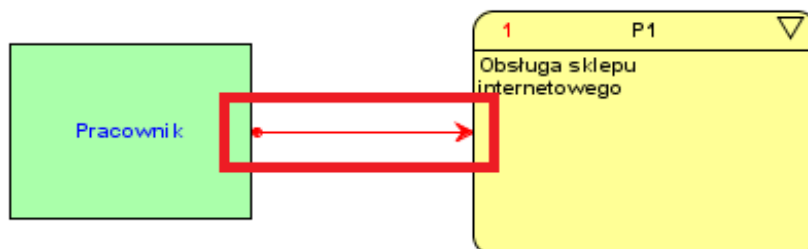
Funkcja / Proces reprezentuje zbiór działań systemu, które są realizowane w konkretnym celu. Może przyjmować i zwracać informacje do innych elementów diagramu.



Rys. 45. Funkcja / proces

8.3.2 Przepływ

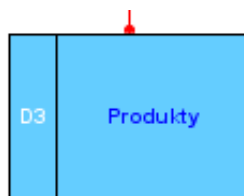
Przepływ prezentuje kierunek przepływu informacji pomiędzy innymi elementami: Agentem, Magazynem, Procesem. Może posiadać atrybut lub encję przekazywaną pomiędzy powiązаныmi obiektami.



Rys. 46. Przepływ

8.3.3 Magazyn danych

Magazyn danych to tymczasowy lub trwały element przechowujący dane. Jest on wykorzystywany przez procesy do pobierania i zapisywania informacji. Może zawierać informację o jednej lub kilku encjach.



Rys. 47. Magazyn danych

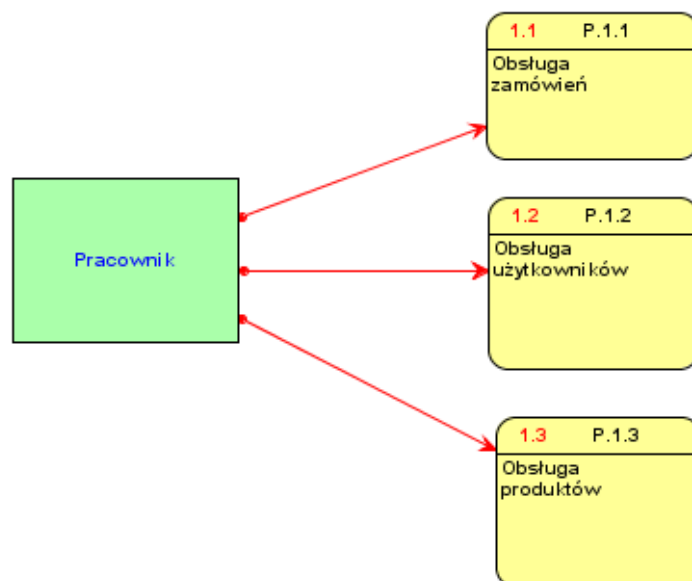
8.3.4 Obiekt zewnętrzny / Agent

Obiekt zewnętrzny / Agent to obiekt, który jest poza granicami systemu. Może być nim inny system, organizacja czy jednostka. Jest odbiorcą lub nadawcą informacji do prezentowanego systemu w diagramie.



Rys. 48. Obiekt zewnętrzny / Agent

Przykładowy diagram DFD, utworzony w programie Data Modeler, pokazano na Rys. 49.



Rys. 49. Przykładowy diagram DFD, utworzony w programie Data Modeler

8.4 Wprowadzenie do tworzenia diagramu DFD

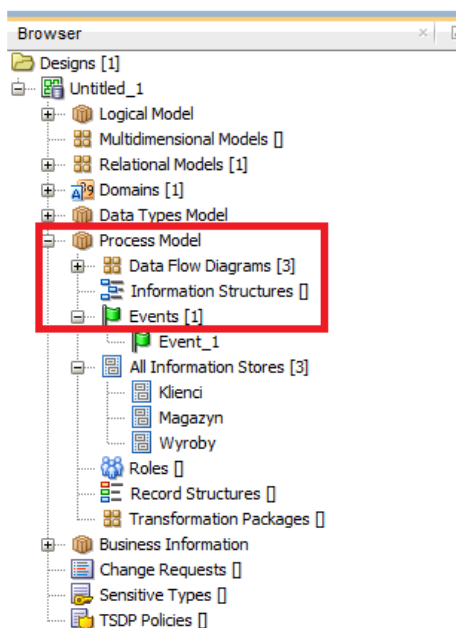
Uproszczony schemat tworzenia diagramu DFD jest następujący:

1. Zdefiniuj procesy, które będą znajdować się na diagramie.
2. Zdefiniuj dane wchodzące do procesów (od aktorów, procesów, magazynów).
3. Zdefiniuj dane wychodzące z procesów (do aktorów, procesów, magazynów).
4. Ustal dane przekazywane w przepływach np. informacje, encje.
5. Określ poziom szczegółowości diagramu oraz ilości procesów elementarnych.

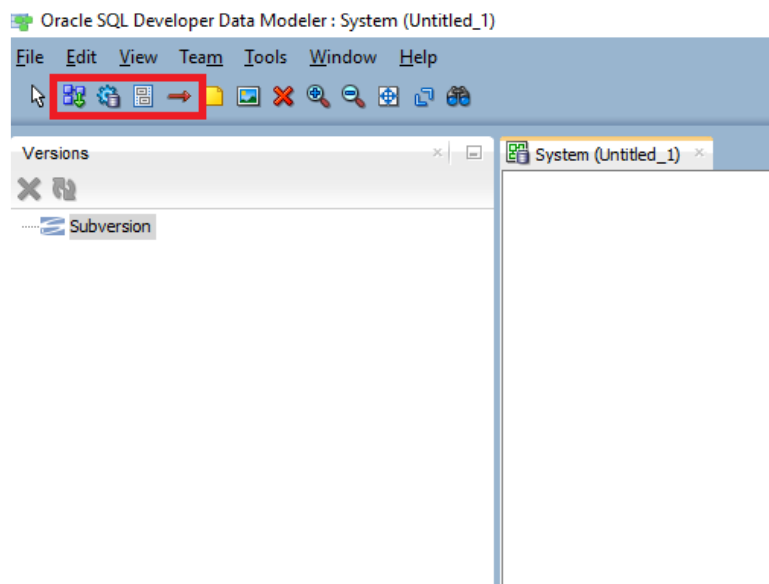
Stworzenia diagramu przepływu danych umożliwia Oracle SQL Developer Data Modeler [5]. Na początku należy stworzyć diagram „Data Flow Diagram” umieszczony w kategorii „Process Model” (zob. Rys. 50). Do stworzenia kompletnego diagramu należy wykorzystać funkcje umożliwiające dodanie podstawowych elementów: Proces, Aktor, Przepływ, Magazyn (zob. Rys. 51). Parametry tych elementów można ustawić w oknie właściwości (zob. Rys. 52). Istotne jest, aby w procesie dekompozycji wykorzystać funkcje w trybie „Złożony”. Umożliwia to wejście do wnętrza procesu i stworzenie dodatkowych podfunkcji. Aby wejść w tryb wewnętrzny funkcji należy użyć opcji „Go To Diagram” (zob. Rys. 53).

Główne kroki tworzenia diagramu przepływu danych są następujące:

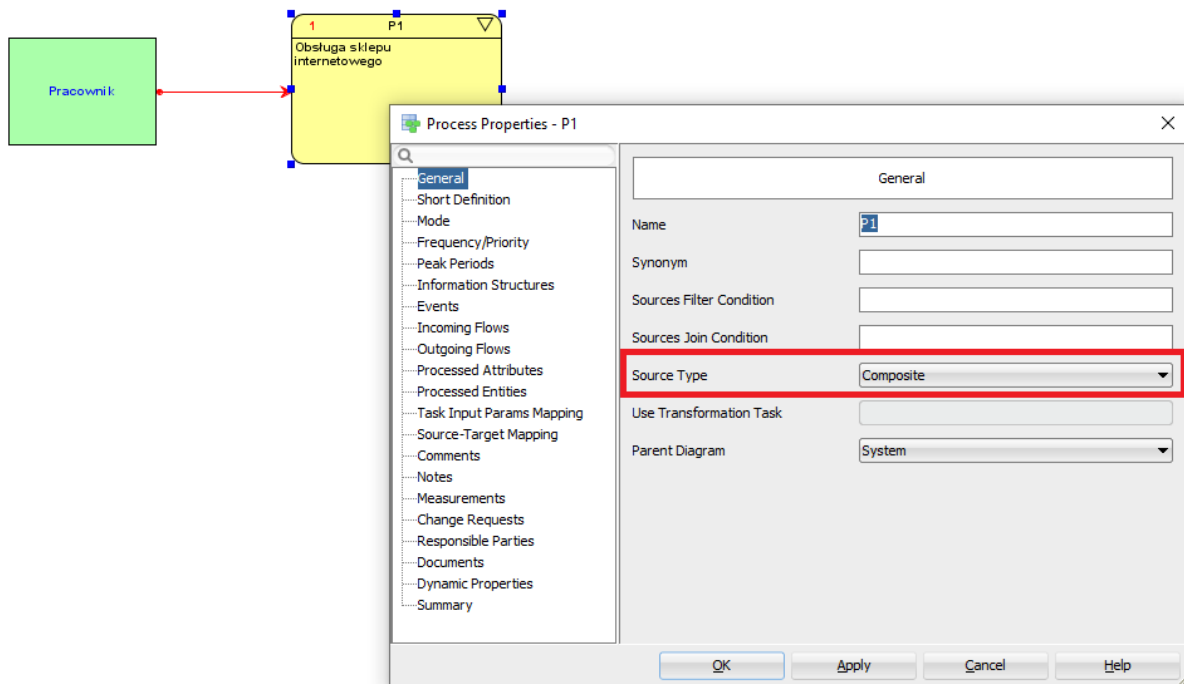
- Określenie głównych procesów w diagramie.
- Określenie obiektów zewnętrznych wschodzących w interakcje z systemem.
- Zdefiniowanie Magazynów danych wymaganych do przeprowadzenia procesom.
- Numeracja oraz opis elementów diagramu.
- Ustalenie kierunku przepływu danych i ich atrybutów.
- Określenie poziomu szczegółowości funkcji (dekompozycja).



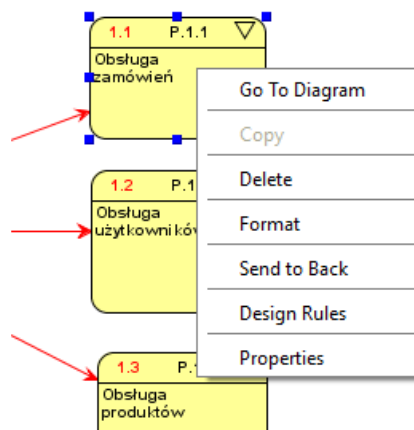
Rys. 50. Drzewo elementów w programie Data Modeler



Rys. 51. Ikonki reprezentujące funkcje dodawania elementów do diagramu DFD w programie Data Modeler



Rys. 52. Okno właściwości procesu w programie Data Modeler



Rys. 53. Opcja „Go To Diagram” służąca do dekompozycji funkcji

9 Szacowanie pracochłonności wykonania projektu

9.1 Wstęp

Szacowanie pracochłonności wykonania projektu jest niezbędne, aby możliwe było określenie ile czasu potrzeba na jego realizację, jeszcze przed przystąpieniem do pracy nad systemem, na etapie specyfikowania wymagań. Nie można wówczas oprzeć się na architekturze systemu, ani na projekcie interfejsu użytkownika, gdyż są one tworzone w późniejszym czasie.

Jedynie informacje zebrane są w postaci wymagań systemu, określonych poprzez przypadki użycia oraz własności produktu. Do tego dysponujemy wiedzą o organizacji wytwarzającej oprogramowanie, w szczególności o zespole, który możemy zaangażować w realizację projektu, oraz o dotychczas wytworzonych przez organizację systemach.

Istnieje wiele metod pozwalających na estymację pracochłonności i kosztów związanych z wytwarzaniem systemów informatycznych. Są to między innymi:

- COCOMO II (skrót od ang. *CO*nstructive *CO*st Model) – bazuje na rozmiarze oprogramowania,
- Metoda delficka – opiera się na wiedzy i intuicji ekspertów,
- Metoda linii kodu,
- Metoda punktów funkcyjnych (ang. *Function Points Method*) – bazuje na projektach ekranów oraz architekturze systemu,
- Metoda punktów przypadków użycia – opiera się na specyfikacji wymagań.

Metoda punktów przypadków użycia oparta jest na:

- Czynniki złożoności środowiska – opisujące organizację/zespół wytwarzający oprogramowanie,
- Czynniki złożoności technicznej – opisujące własności produktu,
- Aktorach,
- Przypadkach użycia (krokach, transakcjach).

Każdy z tych elementów jest przedstawiony w punktach 9.1.1 – 9.1.4, odpowiednio. Ich znajomość jest niezbędna do oszacowania pracochłonności wykonania systemu metodą punktów przypadków użycia, omówioną w podrozdziale 9.2.

9.1.1 Czynniki złożoności środowiska

Na czynniki złożoności środowiska (ang. *ECF* – *Environmental Complexity Factors*) składa się 8 parametrów, którym przydzielane są wagi liczbowe. Im większa waga, tym większy wpływ parametru z nią powiązanego na zmniejszenie pracochłonności przedsięwzięcia.

Wspomniane parametry (czynniki) i odpowiadające im wagi zamieszone są w Tab. 1.

Tab. 16. Czynniki złożoności środowiska

Nazwa czynnika	Waga	Opis
E1 – zaznajomienie z projektem	1,5	Określa, czy zespół jest zaznajomiony z dziedziną problemu i technicznymi aspektami realizowanego projektu. Brane jest

		tu także pod uwagę zaznajomienie zespołu z metodyką, w ramach której realizowany jest projekt.
E2 - doświadczenie w tworzeniu aplikacji	0,5	Określa doświadczenie zespołu w wytwarzaniu oprogramowania.
E3 - Doświadczenie w projektowaniu aplikacji zorientowanych obiektowo	1,0	Określa umiejętność projektowania aplikacji obiektowych i wykorzystania narzędzi do projektowania, np. języka UML.
E4 – Umiejętności głównego analityka	0,5	Określa zdolność głównego analityka do pozyskiwania wymagań od klienta oraz zaznajomienie z wiedzą dziedzinową.
E5 - Motywacja	1,0	Określa motywację zespołu.
E6 - Stabilność wymagań	2,0	Określa czy wymagania nie są narażone na częste zmiany.
E7 - Pracownicy pracujący w niepełnym wymiarze	-1,0	Określa, czy wśród zespołu znajduje się duża liczba pracowników pracujących w niepełnym wymiarze godzin.
E8 - Trudność języka programowania	-1,0	Określa, jak trudny do opanowania jest język programowania, w którym implementowany będzie system.

Warto zauważyć, że czynniki od E1 do E6 mają dodatnie wagi. Oznacza to, że wzrost ich wpływu redukuje pracochłonność. Natomiast czynniki E7 i E8 posiadają ujemne wagi, czyli wraz ze wzrostem ich wpływu zwiększeniu ulegnie także pracochłonność.

Aby wyznaczyć czynnik złożoności środowiska, należy określić wpływ każdego z ośmiu czynników złożoności środowiska w **skali od 0 do 5**. Kolejnym krokiem jest wyliczenie wartości *ECF* korzystając ze wzoru (1).

$$ECF = 1,4 + (-0,03 * \sum w_i * impact_i) \quad (1)$$

gdzie:

w_i – waga i-tego czynnika środowiskowego,

$impact_i$ – wartość z zakresu 1..5, określająca wpływ i-tego czynnika środowiskowego

Warto zauważyć, że w równaniu występuje suma iloczynów wag czynników mnożonych przez czynniki wpływu.

9.1.2 Czynniki złożoności technicznej

Czynniki złożoności technicznej opisują pewne aspekty tworzonego produktu. Podobnie jak w przypadku czynników złożoności środowiska, każdemu z 13 wyszczególnionych czynników złożoności technicznej przypisana jest waga oddająca stopień, w jakim czynnik wpływa na końcowe oszacowanie, jako wartość z zakresu 0..5. Wzrost wpływu danego współczynnika złożoności technicznej powoduje zawsze wzrost końcowego oszacowania pracochłonności.

Wykaz czynników złożoności technicznej, wraz z odpowiadającymi im wagami i opisem zawarty jest w Tab. 17.

Tab. 17. Czynniki złożoności technicznej

Nazwa czynnika	Waga	Opis
T1 – System rozproszony	2,0	Definiuje czy w systemie wymagane jest rozproszone przetwarzanie danych.

T2 - Wydajność	1,0	Określa wydajność systemu, w odniesieniu do czasu odpowiedzi systemu, szybkości przetwarzania.
T3 – Wydajność dla użytkownika końcowego	1,0	Określa istotność wydajności dla końcowego użytkownika, w kontekście jego percepcji.
T4 – Złożone przetwarzanie wewnętrzne	1,0	Określa, czy wymagane są skomplikowane operacje przetwarzania danych, użycie zaawansowanych algorytmów.
T5 – Re-używalność	1,0	Określa, czy kod tworzonej aplikacji będzie w przyszłości wykorzystywany w podobnych projektach.
T6 – Łatwość w instalacji	0,5	Określa sposób instalacji (czy aplikacja musi być zainstalowana przez specjalistów z firmy wytwarzającej oprogramowanie, czy też posiadać będzie przyjazny i łatwy w obsłudze instalator).
T7 – Łatwość użycia	0,5	Określa dostosowanie interfejsu użytkownika do jego potrzeb, wygodę w korzystaniu oraz łatwość uczenia.
T8 – Przenośność	2,0	Określa, czy aplikacja powinna działać w różnych środowiskach.
T9 – Łatwość wprowadzania zmian	1,0	Określa, czy klient będzie chciał w przyszłości rozbudowywać system. Jeśli tak wymusza to stworzenie odpowiedniej architektury.
T10 – Współbieżność	1,0	Określa, czy w aplikacji będzie miało miejsce przetwarzanie współbieżne.
T11 – Specjalne zabezpieczenia	1,0	Określa, czy system będzie wymagał wykorzystanie dodatkowych mechanizmów zabezpieczeń.
T12 – Zależność od zewnętrznych bibliotek	1,0	Określa, w jakim stopniu system bazuje na zewnętrznych bibliotekach. Wykorzystanie bibliotek zmniejsza liczbę funkcji, które należy zaimplementować, jednak niesie za sobą pewne problemy z utrzymaniem zgodności, czy też rozbudowy udostępnianych przez nie funkcjonalności.
T13 – Dodatkowe szkolenia użytkowników	1,0	Określa, czy będą wymagane dodatkowe szkolenia użytkowników, czy też wystarczy stworzona w projekcie dokumentacja.

Oszacowanie pracochłonności związanej z aspektami technicznymi opiera się na współczynniku TCF (Technical Complexity Factor), wyznaczanym z zależności (2):

$$TCF = 0,6 + (-0,01 * \sum w_i * impact_i) \quad (2)$$

gdzie:

w_i – waga i-tego czynnika technicznego,

$impact_i$ – wartość z zakresu 1..5, określająca wpływ i-tego czynnika technicznego

9.1.3 Ocena złożoności aktorów

Jednym z elementów uwzględnianych przy szacowaniu pracochłonności wykonania projektu jest złożoność aktorów występujących w systemie. Pod uwagę brani są aktorzy wyszczególnieni w przypadkach użycia. Wyszczególniono trzy grupy złożoności aktorów:

1. Aktor o złożoności prostej, który komunikuje się z systemem przez API,
2. Aktor o złożoności średniej, który komunikuje się z systemem poprzez protokół (np. HTTP, FTP) lub stanowi źródło danych (pliki, baza danych),
3. Aktor złożony, komunikujący się z systemem poprzez graficzny interfejs użytkownika.

W oparciu o aktorów wyliczany jest współczynnik związany z ich wpływem na pracochłonność systemu, określany jako **niedostosowana waga aktorów** (ang. **UAW** - Unadjusted Actors Weight).

W tym celu każdej grupie złożoności aktorów przypisywana jest waga (współczynnik złożoności grupy aktorów), według następujących zasad:

- Dla aktorów prostych: 1,
- Dla aktorów o średniej złożoności: 2,
- Dla aktorów złożonych: 3.

Współczynnik UAW obliczany jest ze wzoru (3):

$$UAW = \sum n_i * c_i \quad (3)$$

gdzie:

- n_i – liczebność i-tego zbioru złożoności aktorów,
- c_i – współczynnik złożoności i-tego zbioru złożoności aktorów ($c_i \in \{1, 2, 3\}$)

9.1.4 Ocena złożoności przypadków użycia

Złożoność przypadków użycia określana współczynnikiem **UUCW** (ang. *Unadjusted Use Cases Weight*), obliczanym w sposób podobny do złożoności aktorów.

Najpierw wyznaczana jest liczba transakcji w ramach przypadku użycia, czyli kroków lub grup kroków, które mogą zostać wykonane w całości lub w ogóle. W zależności od uzyskanej liczby transakcji dany przypadek użycia kwalifikowany jest do jednej z trzech grup złożoności:

1. Przypadki proste – do tej grupy zaliczane są te przypadki, które mają 3 lub mniej transakcji (współczynnik złożoności wynosi wówczas 5).
2. Przypadki średnie – przypadki z liczbą transakcji od 4 do 7 (współczynnik złożoności wynosi wówczas 10).
3. Przypadki złożone – przypadki z liczbą transakcji powyżej 7 (współczynnik złożoności wynosi wówczas 15).

Wpływ przypadków użycia na prędkość wykonania systemu (UUCW) obliczany jest jako suma iloczynów liczby przypadków użycia w każdej z grup złożoności i odpowiadającego tej grupie współczynnika złożoności (4):

$$UUCW = \sum n_i * c_i \quad (4)$$

gdzie:

- n_i – liczebność i-tego zbioru złożoności przypadków użycia,
- c_i – współczynnik złożoności i-tego zbioru złożoności przypadków użycia ($c_i \in \{5, 10, 15\}$)

9.2 Metoda punktów przypadków użycia

Metoda punktów przypadków użycia (ang. *Use Case Points*, w skrócie UCP) opracowana została w 1993 roku przez Gustava Kainera. Jest to metoda pozwalająca na estymację nakładu pracy oraz kosztów wyprodukowania systemu informatycznego już na etapie zbierania wymagań. Wykorzystuje ona podstawowe elementy techniki obiektowej modelowania systemów informatycznych, takie jak aktor i przypadek użycia oraz charakterystykę przyszłego produktu jak i cechy zespołu pracującego nad rozwiązaniem problemu. Opiera się ona na współczynnikach uzyskanych w oparciu o czynniki złożoności środowiska, czynniki złożoności technicznej, aktorów i przypadki użycia.

Punkty przypadków użycia UCP (ang. *Use Case Points*) obliczane są według zależności (5):

$$UCP = UUCP * TCF * ECF \quad (5)$$

gdzie:

UUCP – niedostosowane punkty przypadków użycia (*ang. Unadjusted Use Case Points*)

TCF – złożoność techniczna (Technical Complexity Factor, zob. 9.1.2)

ECF – złożoności środowiska (Environment Complexity Factor, zob. 9.1.1)

Wartość składowej UUCP (niedostosowane punkty przypadków użycia) obliczana jest jako suma wag aktorów UAW i przypadków użycia UUCW (6):

$$UUCP = UAW + UUCW \quad (6)$$

Oszacowanie pracochłonności liczone jest w osobogodzinach. Wartość taka może zostać uzyskana poprzez przemnożenie punktów przypadków użycia (UCP) przez **współczynnik produktywności PF** (*ang. Productivity Factor*), zdefiniowany jako liczba osobogodzin przypadająca na jeden punkt przypadków użycia.

Autor metody - Gustaw Karner zaproponował przelicznik PF równy 20 roboczogodzinom na jeden punkt przypadków użycia. Z wartości podawanych w literaturze wynika, że współczynnik ten może wahać się w zakresie od 15 do 38 roboczogodzin na jeden punkt przypadków użycia.

Na podstawie obliczonych metoda punktów przypadków użycia osobogodzin pracy nad realizacją projektu, uwzględniając stawki wynagrodzeń pracowników zaangażowanych do jego wykonania oraz dodatkowe koszty, związane z użyciem sprzętów i innych materiałów, obsługą kadrowo-płacowo-finansową, zużyciem mediów, itp. Można oszacować całkowity koszt wykonania projektu.

Proces szacowania kosztów wytworzenia systemu obsługi kursów z użyciem omawianej metody został przedstawiony poniżej.

Przykład

Jako przykład rozpatrzony zostanie system obsługi kursów z Rozdziału 2, którego zakres usług jest następujący (zob. podrozdział 2.3):

1. Moduł katalogu kursów
 - 1.1. instructor
 - 1.1.1. create a description of a course
 - 1.1.2. maintain a bio about myself
 - 1.2. site visitor
 - 1.2.1. view a detailed page about a course
 - 1.2.2. view the bio of an instructor who create a course
 - 1.2.3. browse a catalog of all courses
 - 1.2.4. pick the right course
 - 1.2.5. see a list of other related courses when viewing a course
2. Moduł zarządzania kontem
 - 2.1. site visitor
 - 2.1.1. create an account
 - 2.2. participant
 - 2.2.1. track my progress
 - 2.2.2. manage my account
 - 2.2.3. view a list of all courses I've taken and the PDUs each earned me

Do obliczenia ilości osobogodzin potrzebnych na wykonanie projektu wygodnie jest posłużyć się macierzą odpowiedzialności, opracowaną dla tego systemu (Tab. 2).

Dla każdego z modułów należy wyznaczyć punkty przypadków użycia (UCP). Potrzebujemy do tego wartości następujących składników (wzór 5):

- niedostosowane punkty przypadków użycia (UUCP), będące sumą współczynników UAW (wzór 3) i UUCW (wzór 4),
- współczynnik złożoności technicznej (TCF),
- współczynnik złożoności środowiskowej (współczynnik ECF).

Wyznaczanie UUCP dla modułu katalogu kursów

Złożoność każdego z 2 aktorów w module katalogu kursów wynosi 3, gdyż korzystają oni z interfejsu użytkownika (zob. punkt 9.1.3). Wobec tego współczynnik UAW dla tego modułu wynosi 6 ($= 2 * 3$).

Współczynnik UUCW został obliczony w oparciu o pokazane w Tab. 18 oszacowanie typów występujących w module katalogu kursów przypadków użycia (komórki zaznaczone na niebiesko). Uzyskana w wyniku zastosowania zależności (5) wartość wynosi 60.

Tab. 18. Przypadki użycia i odpowiadające im wagi w module katalogu kursów

Przypadek użycia	typ	waga
create a description of a course	złożony	15
maintain a bio about myself	średni	10
view a detailed page about a course	złożony	15
browse a catalog of all courses	średni	10
pick the right course	prosty	5
see a list of other related courses when viewing a course	prosty	5
	UUCW	60

Wobec tego współczynnik UUCP dla tego modułu wyniesie 66 (60 + 6)

Wyznaczanie ECF dla modułu katalogu kursów

Na potrzeby obliczenia współczynnika ECF należy ocenić wpływ każdego z czynników E1..E8 (Tab. 16), w skali 0..5. Przy określeniu tego wpływu w sposób pokazany w Tab. 19, ze wzoru (1) otrzymujemy wartość współczynnika ECF równą 1,505.

Tab. 19. Ocena wpływu czynników środowiskowych dla modułu katalogu kursów

Czynnik	Symbol	Waga	Wartość	Skala
Zaznajomienie członków zespołu z projektem, metodyką, procesem	E1	1,5	0	0,0
Doświadczenie zespołu w tworzeniu aplikacji	E2	0,5	0	0,0
Doświadczenie zespołu w programowaniu obiektowym	E3	1	2	2,0
Umiejętności głównego analityka	E4	0,5	1	0,5
Motywacja zespołu	E5	1	1	1,0
Stabilność wymagań	E6	2	0	0,0
Udział pracowników zewnętrznych lub na umowach śmieciowych	E7	-1	4	-4,0
Trudność języka programowania	E8	-1	3	-3,0
			ECF	1,505

Wyznaczanie TCF dla modułu katalogu kursów

Na potrzeby obliczenia współczynnika TCF należy ocenić wpływ każdego z czynników T1..T13 (Tab. 17), w skali 0..5. Przy określeniu tego wpływu w sposób pokazany w Tab. 20, ze wzoru (2) otrzymujemy wartość współczynnika TCF równą 1,15.

Tab. 20. Ocena wpływu czynników technicznych dla modułu katalogu kursów

Czynnik	Symbol	Waga	Wartość	Skala
System rozproszony	T1	2	4	8,0
Wydajność	T2	1	4	4,0
Sprawność działania dla użytkownika końcowego	T3	1	4	4,0
Złożone przetwarzanie wewnętrzne	T4	1	5	5,0
Wymaganie ponownej używalności kodu	T5	1	5	5,0
Łatwość instalacji	T6	0,5	5	2,5
Łatwość użytkowania	T7	0,5	5	2,5
Przenośność	T8	2	3	6,0
Łatwość wprowadzania zmian	T9	1	5	5,0
Współbieżność	T10	1	3	3,0
Specjalne zabezpieczenia i mechanizmy ochrony	T11	1	2	2,0
Zależność od systemów zewnętrznych	T12	1	4	4,0
Konieczność specjalnego przeszkolenia personelu	T13	1	4	4,0

TCF	1,15
-----	------

Punkty przypadków użycia dla modułu katalogu kursów

Dla uzyskanych wyżej dla modułu katalogu kursów wartości współczynników UUCP, ECF i TCW obliczona ze wzoru (5) wartość współczynnika UCP wynosi 114,2295.

W analogiczny sposób obliczana jest liczba punktów użycia dla modułu zarządzania kontem.

Wyznaczanie UUCP dla modułu zarządzania kontem

Złożoność każdego z 2 aktorów w module zarządzania kontem wynosi 3, gdyż korzystają oni z interfejsu użytkownika (zob. punkt 9.1.3). Wobec tego współczynnik UAW dla tego moduły wynosi 6 ($= 2 * 3$).

Współczynnik UUCW został obliczony w oparciu o pokazane w Tab. 21 Tab. 18 oszacowanie typów występujących w module katalogu kursów przypadków użycia (komórki zaznaczone na niebiesko). Uzyskana w wyniku zastosowania zależności (5) wartość wynosi 50.

Tab. 21. Przypadki użycia i odpowiadające im wagi w module zarządzania kontem

Przypadek użycia	typ	waga
create an account	złożony	15
track my progress	złożony	15
manage my account	prosty	5
view a list of all courses I've taken and the PDUs each earned me	złożony	15
	UUCW	50

Wobec tego współczynnik UUCP dla tego modułu wyniesie 56 ($50 + 6$)

Wyznaczanie ECF dla modułu zarządzania kontem

Na potrzeby obliczenia współczynnika ECF należy ocenić wpływ każdego z czynników E1..E8 (Tab. 16), w skali 0..5. Przy określeniu tego wpływu w sposób pokazany w Tab. 22 ze wzoru (1) otrzymujemy wartość współczynnika ECF równą 1,505.

Tab. 22. Ocena wpływu czynników środowiskowych dla modułu zarządzania kontem

Czynnik	Symbol	Waga	Wartość	Skala
Zaznajomienie członków zespołu z projektem, metodyką, procesem	E1	1,5	0	0,0
Doświadczenie zespołu w tworzeniu aplikacji	E2	0,5	0	0,0
Doświadczenie zespołu w programowaniu obiektowym	E3	1	2	2,0
Umiejętności głównego analityka	E4	0,5	1	0,5
Motywacja zespołu	E5	1	1	1,0
Stabilność wymagań	E6	2	0	0,0
Udział pracowników zewnętrznych lub na umowach śmieciowych	E7	-1	4	-4,0

Trudność języka programowania	E8	-1	3	-3,0
			ECF	1,505

Wyznaczanie TCF dla modułu zarządzania kontem

Na potrzeby obliczenia współczynnika TCF należy ocenić wpływ każdego z czynników T1..T13 (Tab. 17), w skali 0..5. Przy określeniu tego wpływu w sposób pokazany w Tab. 23, ze wzoru (2) otrzymujemy wartość współczynnika TCF równą 1,12.

Tab. 23. Ocena wpływu czynników technicznych dla modułu zarządzania kontem

Czynnik	Symbol	Waga	Wartość	Skala
System rozproszony	T1	2	4	8,0
Wydajność	T2	1	4	4,0
Sprawność działania dla użytkownika końcowego	T3	1	4	4,0
Złożone przetwarzanie wewnętrzne	T4	1	2	2,0
Wymaganie ponownej używalności kodu	T5	1	5	5,0
Łatwość instalacji	T6	0,5	5	2,5
Łatwość użytkowania	T7	0,5	5	2,5
Przenośność	T8	2	3	6,0
Łatwość wprowadzania zmian	T9	1	5	5,0
Współbieżność	T10	1	3	3,0
Specjalne zabezpieczenia i mechanizmy ochrony	T11	1	2	2,0
Zależność od systemów zewnętrznych	T12	1	4	4,0
Konieczność specjalnego przeszkolenia personelu	T13	1	4	4,0
			TCF	1,12

Punkty przypadków użycia dla modułu zarządzania kontem

Dla uzyskanych wyżej dla modułu zarządzania wartości współczynników UUCP, ECF i TCW obliczona ze wzoru (5) wartość współczynnika UCP wynosi 94,3936.

Punkty przypadków użycia dla systemu obsługi kursów

Współczynnik UCP dla systemu obsługi kursów jest sumą współczynników UCP obliczonych dla każdego z modułów. W omawianym przykładzie uzyskamy więc wartość 208,6231.

Przy założeniu, że współczynnik produktywności $PF = 20$ uzyskamy dla tego systemu liczbę osobogodzin potrzebnych na jego wytworzenie równą 4 172,46.

Szacowanie kosztów wytworzenia systemu

Przy założeniu, że nad systemem pracować będzie zespół złożony z 6 osób, zatrudnionych w pełnym wymiarze czasu pracy (po 8 godzin dziennie). Wobec tego na jednego pracownika przypada 695,4 godzin, co oznacza, że na wykonanie systemu potrzebnych będzie 87 dni, czyli ok. 17,5 tygodni. Na oszacowanie kosztów całkowitych przyjąć więc można, że praca zajmie 4 miesiące. Szacunkowa czasochłonność projektu została zebrana w Tab. 24.

Tab. 24. Szacunkowa czasochłonność wykonania systemu obsługi kursów

Parametr	Wartość	Uwagi
----------	---------	-------

Osobogodziny	4 172,46	
Dni	87	8 godzin pracy, 6 osób w zespole
Tygodnie	17,5	5 dni / tydzień
Miesiące	4	22 dni roboczych / miesiąc

Powyższe wartości dotyczą kosztów osobowych. Dodatkowym kosztem jest sprzęt i materiały niezbędne do pracy. Niezbędne jest zapewnienie każdemu pracownikowi komputera, do tego potrzebny jest serwer oraz miejsce, w którym wykonywany będzie system. Na potrzeby oszacowania związanych z tym kosztów przyjmijmy wartości przedstawione w Tab. 25.

Tab. 25. Szacunkowe wartości kosztów dodatkowych wykonania systemu obsługi kursów

Rodzaj kosztu	Wartość brutto / miesiąc dla 1 osoby
Leasing laptopa ¹	229,00 zł
Hosting serwera ²	605,16 zł (6 miesięcy)
Biurko dedykowane coworking ³	738,00 zł
Obsługa kadrowo – księgowa ⁴	300 zł (spółka cywilna, 40 dokumentów/miesiąc) + 50 zł za jednego pracownika

Dla omawianego systemu otrzymamy więc wartości dodatkowych kosztów jego wytworzenia, obliczone dla stawek z Tab. 25, zamieszczone w Tab. 26.

Tab. 26. Pozaosobowe koszty wykonania systemu obsługi kursów

Rodzaj kosztu	Wartość brutto / czas projektu dla 6 osób
Leasing laptopa (dla 6 osób na 5 m-cy)	6 870 zł (5 miesięcy)
Hosting serwera (na 6 m-cy)	605,16 zł (6 miesięcy)
Biurko dedykowane coworking (dla 6-ciu osób na 5 m-cy)	22 140 zł (5 miesięcy)
Obsługa kadrowo – księgowa (5 miesięcy)	76 500 zł

Wobec tego łączny koszt pozaosobowy projektu wyniesie 16 115,16 zł netto. Przy stawce VAT wynoszącej 23% otrzymamy koszt brutto = 130 521,65 zł.

¹ Katarzyna Rostkowska *Leasing laptopa - jaki jest koszt i czy to się opłaca?* www.bankier.pl 2019-01-31

² Cennik home.pl Cloud VPS L na dzień 28.04.2021

³ Cennik mindwork na dzień 28.04.2021

⁴ Cennik taxland na dzień 28.04.2021

Całkowity koszt systemu jest sumą kosztów osobowych i pozaosobowych. Uwzględniając koszt pracy 1 pracownika, zatrudnionego na umowę o pracę ze stawką brutto równą średniej krajowej (w roku 2023 przewidywana ma to być 6 935 zł⁵), miesięczny koszt jego pracy (łącznie z kosztami pracodawcy i z założeniem uczestniczenia pracownika w PPK) wyniesie 8 459,33 zł. Wobec tego koszt wynagrodzeń 6-ciu pracowników, pracujących w projekcie przez 4 miesiące równy będzie 203 023,92 zł.

Sumując to z kosztami pozaosobowymi otrzymujemy szacunkowy koszt wykonania systemu w wysokości **333 545,57 zł**.

⁵ Podatnik.info <https://www.podatnik.info/publikacje/przecietne-wynagrodzenie-2022-2023,6136dd>

Bibliografia

- [1] I. Sommerville, Inżynieria oprogramowania, PWN, ,2021.
- [2] „UMLetino,” [Online]. Available: <https://www.umletino.com/umletino.html>. [Data uzyskania dostępu: 17 10 2022].
- [3] „Mountain Goat Software,” Mountain Goat Software, [Online]. Available: <https://www.mountaingoatsoftware.com/>. [Data uzyskania dostępu: 17 10 2022].
- [4] Ż. Golonka, „Model związków encji,” Encyklopedia Zarządzania, [Online]. Available: https://mfiles.pl/pl/index.php/Model_zwi%C4%85zk%C3%B3w_encji.
- [5] Oracle, „Data Modeling with Oracle SQL Developer,” Oracle, 2023. [Online]. Available: <https://www.oracle.com/pl/database/sqldeveloper/technologies/sql-data-modeler/>. [Data uzyskania dostępu: 25 marzec 2023].
- [6] Z. Rosiek, „Mapowanie obiektowo-relacyjne (ORM) - czy tylko dobra idea?,” *Zeszyty Naukowe*, pp. 99 - 112, 2009.
- [7] M. Ochodek, J. Nawrocki i K. Kwarcia, „Simplifying effort estimation based on Use Case Points,” *2. Information and Software Technology*, tom 53, nr 3, pp. 200-213, Marzec 2011.

Słowniczek

Encyjny komponent EJB reprezentuje dane, które są trwale przechowywane w systemie bazy danych. Cechuje się długim czasem życia, pozwala na atomowe, transakcyjne modyfikacje bazy danych, jest współdzielony przez wielu klientów i dostępny dla wielu sesji. Dla klienta komponent encyjny to komponent reprezentujący w sposób obiektowy pewne dane (encje) zapisane w pamięci trwałej (np. w bazie danych) lub dane generowane przez działającą aplikację.

ERM (Entity Relationship Model) – model związków encji

ERD (Entity Relationship Model) – diagram związków encji